



National Institute of Applied Sciences and Technology

CARTHAGE UNIVERSITY

Graduation Project

Specialty : Software Engineering

Evidence Graph Construction for Reliable Question Answering over Large-Scale Scientific Literature

Presented by

Ilef Chebil

INSAT Supervisor : **Prof. Dr. Sonia BOUZIDI**

Company Supervisor : **Prof. Dr. Michael FÄRBER**

Presented on : **–/06/2026**

JURY

M. President FLEN (President)

Ms. Reviewer FLENA (Reviewer)

Academic Year : 2025/2026



National Institute of Applied Sciences and Technology

CARTHAGE UNIVERSITY

Graduation Project

Specialty : Software Engineering

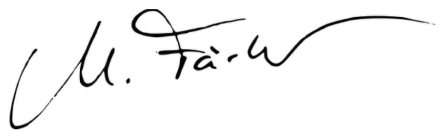
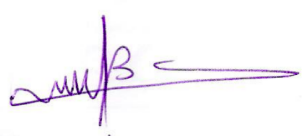
Evidence Graph Construction for Reliable Question Answering over Large-Scale Scientific Literature

Presented by

Ilef Chebil

Hosted by



<i>Supervisor at SCADS.AI</i> Prof. Dr. Michael FÄRBER	<i>Supervisor at INSAT</i> Prof. Dr. Sonia BOUZIDI
 Date:	 Date:

Academic Year : 2025/2026

Dedication

To my mother Amel, the person I love most, you are the sun of my life. Without you, I would be in complete darkness. Thank you for being one of my strongest motivators in this life and for giving me the strength to keep going. Anything I am or will ever be, I owe it to you.

To my father Fethi, my first teacher, the person who gave me my name, taught me my first letters and words, and read me my first book. I still carry with me the lessons and principles you taught me when I was young.

To my sister Nouha, you are the one I always look up to, my safe place and comfort person. In this life, there are many things to be grateful for, and the greatest among them is having a sister. To my brother Ghaith, my childhood best friend and my first-ever friend. My childhood was filled with great adventures thanks to you.

To my grandmother Omi Salha, may she rest in peace. Years pass, but I think of you always and miss you deeply. I will carry your memory and love with me forever. Thank you for giving me the warmest motherly love.

To all my family, thank you so much. You are my pillar and my strength. Nothing in this life is more important to me than you.

To my partner Youssef, this journey wouldn't have been the same without you. I am truly grateful that we shared this college experience together from day one. Through it all, your support and belief in me pushed me further.

Lastly, to my teachers, to everyone who has ever taught me anything, whether small or big, and to anyone who has believed in me and supported me, thank you from the bottom of my heart.

Acknowledgments

I would like to express my gratitude to everyone who supported me throughout this journey.

First and foremost, I would like to thank my academic supervisor at INSAT, Dr. Sonia Bouzidi, for her guidance and continuous support. Her feedback and encouragement were invaluable throughout this work.

I also extend my heartfelt thanks to my supervisor at ScaDS.AI, Dr. Michael Färber, for his valuable insights, constructive feedback, and thoughtful advice that greatly enriched my experience.

Finally, I would like to express my appreciation to the honorable members of the jury for dedicating their time to evaluate this work.

Table of Contents

List of Figures	vi
List of Tables	vii
Résumé	viii
Abstract	ix
General Introduction	1
I Project Context	2
1 Introduction of the Host Organization	2
1.1 Presentation of TUD Dresden University of Technology	3
1.2 Presentation of ScaDS.AI Dresden/Leipzig	3
2 Project definition	4
2.1 General Context	4
2.2 Problem Statement and Motivation	4
3 Objectives	6
4 Methodology and Project Timeline	6
4.1 Work Methodology	6
4.1.1 Adapting the CRISP-DM Methodology	6
4.1.2 Choice Justification	7
4.2 Project Timeline	8
II Business Understanding and Technical Concepts	10
1 Domain-Related Concepts	11
1.1 Structure of Scientific Documents	11
1.2 Scientific Evidence, Claims, and Citations	11
1.3 Citation Intent	11
2 Technical Foundations	12
2.1 Large Language Models (LLMs)	12
2.2 Retrieval-Augmented Generation (RAG)	13
2.2.1 Lexical Sparse Retrieval	13
2.2.2 Semantic Dense Retrieval	14

2.2.3	Hybrid Retrieval and Reranking	14
2.3	Natural Language Inference (NLI)	15
2.4	Multi-Agent Systems and Orchestration	15
2.5	Property Graphs for Knowledge Representation	15
2.6	High-Performance Computing (HPC) Infrastructure	16
3	Related Work	17
3.1	Large-Scale Scholarly Datasets	18
3.2	Multi-Agent Retrieval-Augmented Generation	18
3.3	Citation-Grounded Answer Generation	19
3.4	Scientific Question Answering Systems	19
III Data Understanding and Preparation		22
1	Dataset Source	23
1.1	The UnarXive 2024 Corpus	23
1.2	Raw Data Format and Schema	24
1.3	Dataset Limitations and Quality Issues	25
2	Index Creation Pipeline	25
2.1	Paper Normalization and Preprocessing	25
2.2	Chunking and Embedding Generation	26
2.3	Vector Database Ingestion on HPC	27
2.4	Post-Indexing Enrichment	28
2.5	End-to-End Pipeline Overview	29
3	Exploratory Analysis	30
3.1	Index Schema and Payload Structure	30
3.2	Index Statistics and Coverage	31
3.3	Temporal Coverage of the Index	32
3.4	IMRaD Structural Label Distribution	32
IV System Design and Architecture		34
1	Overview of EviGraph-R Agentic Architecture	35
1.1	High-Level Architecture	35
1.2	Orchestration, Structured Reasoning, and Observability	35
2	Hybrid Retrieval Node Design	36
2.1	BGE-M3 Sparse Vector Search	37
2.2	BGE-M3 Dense Vector Search	37

2.3	Reciprocal Rank Fusion (RRF)	38
2.4	Cross-Encoder Reranking	38
2.5	Section Filtering and Budget-Weighted Scoring	38
3	Agent Roles and Responsibilities	39
3.1	Query Decomposer Agent	39
3.2	Graph Builder Agent	40
3.3	Judge Agent	43
3.4	Answer Generator Agent	43
4	End-to-End Walkthrough	45
4.1	Query Decomposition	45
4.2	Sub-query Retrieval	46
4.3	Evidence Graph Construction	47
4.4	Claim Verification	47
4.5	Answer Generation and Citations	49
V	System Evaluation and Integration	50
1	Index and Retrieval Evaluation	51
1.1	Experimental Setup	51
1.2	Retrieval Metrics	52
1.3	Retrieval Performance and Interpretation	53
2	System Evaluation	55
2.1	Evaluation Framework	55
2.2	Evaluation Metrics	57
2.3	Ablation Study	57
2.4	Baselines Evaluation	59
2.5	Results and Interpretation of Findings	59
3	Software Integration	60
3.1	Backend: FastAPI Service	60
3.2	System Monitoring: Langfuse Tracing	62
3.3	Frontend: ReactJS Interface	62
3.4	Pip Package	67
	Conclusion and Perspectives	68
	References	69

List of Figures

I.1	Logo of TUD Dresden University of Technology.	3
I.2	Logo of ScaDS.AI Dresden/Leipzig.	3
I.3	The CRISP-DM model (Cross Industry Standard Process for Data Mining) [1]. .	7
I.4	GANTT diagram of the project phases and timeline.	9
II.1	The Transformer Model Architecture [2]	12
II.2	Overview of RAG architecture [3].	13
II.3	Example of a property graph representing bibliographic information [4].	16
II.4	Architectural overview of the Capella HPC cluster at ZIH, TU Dresden [5]. . . .	17
III.1	Annual number of publications in unarXive 2024, broken down by discipline. [6].	23
III.2	End-to-end overview of the EviGraph-R index creation pipeline.	30
III.3	Number of indexed chunks per publication year	32
III.4	IMRaD label distribution across all 69 million indexed chunks after post-processing	33
IV.1	High-level architecture of the EviGraph-R multi-agent system.	36
IV.2	Internal flow of the Decomposer Agent.	40
IV.3	Internal flow of the Graph Builder Agent.	42
IV.4	Verification pipeline of the Judge Agent.	44
IV.5	Flow of the Answer Generator Agent.	45
IV.6	Minimal evidence graph for the running example.	48
V.1	Retrieval performance of EviGraph-R vs. SQuAI across three domains.	53
V.2	The EviGraph-R interactive API documentation at /docs.	61
V.3	GET /health response confirming Qdrant reachability and the loaded embedding model.	61
V.4	Langfuse trace showing per-agent latency for a sample EviGraph-R query.	62
V.5	Landing screen with query cards, collapsible sidebar, and the input bar.	63
V.6	Pipeline progress tracker during a live query.	63
V.7	Result panel after query completion.	64
V.8	Citation side drawer displaying chunk text, paper metadata and IMRaD.	65
V.9	Interactive evidence graph rendered with Cytoscape.js.	66
V.10	Settings panel exposing Top K, score threshold, toggle, and IMRaD section filter.	66
V.11	EviGraph-R listing on PyPI.	67

List of Tables

I.1	Brief comparison of KDD, SEMMA, and CRISP-DM.	8
II.1	Key hardware specifications of the Capella GPU cluster at ZIH, TU Dresden. . .	17
II.2	Comparison of large-scale scholarly datasets and retrieval indexes. ✓ = available, × = not available, – = not applicable.	18
II.3	Comparison of scientific QA systems across key dimensions. ✓ = yes, × = no, ~ = partial.	20
III.1	BAAI/bge-m3 embedding model specifications.	27
III.2	Qdrant collection configuration for unarxive_chunks.	28
III.3	IMRaD label distribution after post-indexing normalisation.	29
III.4	Payload schema of the unarxive_chunks Qdrant collection.	31
III.5	Final index statistics for unarxive_chunks.	31
IV.1	Decomposer output for the running example query.	46
IV.2	Representative retrieved chunks per sub-query.	46
V.1	Evaluation corpus: nine topical clusters across three scientific domains.	52
V.2	Question types used in the evaluation.	52
V.3	Primary retrieval metrics.	52
V.4	Supplementary retrieval metrics.	53
V.5	Primary retrieval metrics at $k = 1, 5, 10$ for EviGraph-R vs. SQuAI (overall). . .	54
V.6	Hit@ k and MRR@10 broken down by scientific domain and question type. . . .	54
V.7	Domain distribution of the benchmark dataset.	55
V.8	NFQ style distribution across the benchmark dataset.	56
V.9	Ablation and baselines results.	60

Résumé

Ce rapport présente un stage mené à l'Université technique de Dresde (TUD), au sein de ScaDS.AI, un centre de recherche de référence en Allemagne. Le projet s'est structuré autour de trois grands axes : la construction d'un index de recherche à grande échelle, la conception d'un système de raisonnement multi-agent, et l'élaboration d'un protocole d'évaluation rigoureux.

Le premier axe a abouti à l'indexation du corpus unarXive à grande échelle, couvrant plus de 2,2 millions d'articles académiques. Les passages sont représentés par des vecteurs denses et creux, stockés dans une base de données vectorielle, puis enrichis par des pipelines de post-traitement assurant la résolution des citations et la normalisation des titres de sections.

Sur cette base, le deuxième axe a porté sur le développement d'un système de raisonnement multi-agent. Des agents fondés sur des LLM décomposent les questions complexes en sous-requêtes ciblées, et pilotent une recherche hybride dense-creuse, une fusion par rang réciproque et un reclassement par cross-encoder. Les passages récupérés sont ensuite structurés en graphe de preuves, où les affirmations scientifiques sont rendues explicites, reliées à leurs sources, et utilisées comme de vérifiables unités de raisonnement. Un agent juge valide ces affirmations avant de transmettre uniquement les preuves validées à un générateur de réponses capable de citer ses sources.

Pour évaluer la fiabilité de cette architecture, le troisième axe a consisté à construire un benchmark d'évaluation synthétique de 268 questions. Celui-ci soumet le système à l'épreuve à travers quatre catégories de questions couvrant sept styles de formulation et cinq métriques complémentaires.

Ce travail établit ainsi un pont entre la recherche documentaire scientifique à grande échelle et un raisonnement fidèle, ancré dans les preuves. Pour en faire profiter la communauté de recherche, EviGraph-R est mis à disposition avec un index public, une démo interactive et un package installable.

Mots-clés : question-réponse scientifique, unarXive, graphe de preuves, systèmes multi-agents, recherche hybride dense-creuse, vérification des affirmations, ancrage citationnel.

Abstract

This report documents an internship experience at Dresden University of Technology (TUD), ScaDS.AI, a leading research institution in Germany. The project centered on three objectives: building a large-scale retrieval index, designing a multi-agent reasoning system, and constructing a rigorous evaluation benchmark.

The first objective resulted in indexing the unarXive dataset at scale, covering over 2.28 million academic papers. Chunks are embedded with dense and sparse vectors, stored in a vector database, and enriched through post-processing pipelines for citation resolution and section-title normalization.

Building on this foundation, the second objective was the development of a multi-agent reasoning system. LLM-based agents decompose complex questions into targeted sub-queries, and guide hybrid dense-sparse retrieval, Reciprocal Rank Fusion, and cross-encoder reranking. Retrieved chunks are then organized into an evidence graph, where scientific claims are made explicit, linked to source chunks and papers, and used as verifiable reasoning units. A judge agent verifies these claims before forwarding only validated evidence to a citation-aware answer generator.

To assess the reliability of this architecture, the third objective focused on constructing a synthetic evaluation benchmark of 268 questions. The benchmark stress-tests the system across four question categories spanning seven question styles and five complementary metrics.

Overall, this work bridges the gap between large-scale scientific retrieval and faithful, evidence-grounded reasoning. To support the research community, EviGraph-R is released with a public index, an interactive demo system and an installable package.

Keywords: scientific question answering, unarXive, evidence graph, multi-agent systems, hybrid dense-sparse retrieval, claim verification, citation grounding.

General Introduction

The rapid growth of scientific literature has made it increasingly difficult for researchers to find, connect, and verify evidence across large collections of papers. This challenge is central to scientific question answering, where answers must be accurate, traceable, and grounded in reliable sources.

Existing approaches to scientific question answering share two fundamental limitations. The first is structural: systems that rely on retrieval, whether through RAG, hybrid pipelines, or agentic decomposition, treat retrieved evidence as raw text to be injected into a language model, with no explicit representation of how claims, citations, and findings relate to one another. The result is answers that may cite sources without genuinely reasoning over them. The second is at the data level: large corpora such as arXiv provide access to millions of papers, but without the infrastructure needed to actually use them, no chunk-level index, no section-aware representation, and no citation resolution layer. Together, these two gaps separate having access to scientific literature from being able to produce answers that are faithful, traceable, and transparently grounded.

To address those gaps, this work presents EviGraph-R, a multi-agent system for reliable question answering over large-scale scientific literature. Built on the unarXive corpus of over 2.28 million papers, the system combines structure-aware indexing with a LangGraph-orchestrated reasoning pipeline. Specialized agents decompose incoming queries, perform hybrid dense-sparse retrieval with reranking, and organize retrieved evidence into an evidence property graph where scientific claims are made explicit and linked to their sources. A judge agent then verifies each claim before only validated evidence reaches the answer generator, producing responses that are both fluent and transparently grounded.

This report is organized into five chapters. The first introduces the host organization, project scope, and methodology. The second covers the technical foundations, including RAG paradigms, multi-agent systems, and related work. The third describes the unarXive dataset, its indexing pipeline, and a corpus-level exploratory analysis. The fourth details EviGraph-R’s architecture, from each agent’s design to an end-to-end walkthrough. The fifth and final chapter presents the evaluation framework, ablation study, and software integration.

Chapter I

Project Context

Summary

1	Introduction of the Host Organization	2
1.1	Presentation of TUD Dresden University of Technology	3
1.2	Presentation of ScaDS.AI Dresden/Leipzig	3
2	Project definition	4
2.1	General Context	4
2.2	Problem Statement and Motivation	4
3	Objectives	6
4	Methodology and Project Timeline	6
4.1	Work Methodology	6
4.2	Project Timeline	8

Introduction

In this chapter, we first introduce the host organization and the environment in which the internship took place. We then present the project context, the main problem, and the motivation behind the proposed work. Finally, we summarize the project objectives, methodology, and timeline followed throughout the internship, setting the stage for the following detailed discussions.

1 Introduction of the Host Organization

This project was carried out at TUD Dresden University of Technology, within the Faculty of Computer Science and in collaboration with the Center for Scalable Data Analytics and Artificial Intelligence, ScaDS.AI Dresden/Leipzig.

1.1 Presentation of TUD Dresden University of Technology

TUD Dresden University of Technology is one of Germany’s leading technical universities and one of the largest institutions of higher education in the country. It is organized into five Schools and 17 Faculties, offering 119 degree programs across a wide range of disciplines. This structure reflects the university’s strong interdisciplinary profile and its role as a major center for education, research, and innovation in Germany [7].

TUD has held the title of University of Excellence since 2012. In March 2026, this status was confirmed once again within the German Excellence Strategy. This recognition highlights the university’s strong research performance, international orientation, and long-term commitment to scientific excellence [8].



Figure I.1 – Logo of TUD Dresden University of Technology.

1.2 Presentation of ScaDS.AI Dresden/Leipzig

ScaDS.AI Dresden/Leipzig, the Center for Scalable Data Analytics and Artificial Intelligence, is a national AI competence center with locations in Dresden and Leipzig. The center focuses on research in artificial intelligence, big data, and data science, with particular attention to scalable, reliable, and responsible data-driven methods.

The center is institutionally funded by the Federal Ministry of Research, Technology and Space and the Free State of Saxony. Today, ScaDS.AI provides an interdisciplinary research environment that connects AI methods, scalable data engineering, knowledge management, and real-world scientific applications [9].



Figure I.2 – Logo of ScaDS.AI Dresden/Leipzig.

2 Project definition

To define and understand the project, we need to provide the general context, the problem and the motivation.

2.1 General Context

The volume of scientific literature available today has grown far beyond what researchers can manually read, synthesize, and cross-reference. Preprint repositories such as arXiv [10] now host millions of full-text papers spanning virtually every scientific discipline. This is a remarkable opportunity, but it comes with a real problem: the knowledge researchers need often exists somewhere in that vast collection, yet locating it, connecting the right pieces, and actually verifying a claim remains surprisingly hard.

This difficulty has pushed the research community toward automated scientific question answering. The idea is to help users find relevant evidence across large paper collections and turn it into accurate, well-supported answers. Such systems could speed up literature reviews, lower the barrier to scientific knowledge, and make research more accessible. The catch is that none of this matters if the answers cannot be trusted.

Recent progress in Large Language Models and Retrieval-Augmented Generation has made this goal feel more reachable. These approaches can pull relevant documents and generate coherent responses, but fluency alone is not enough for scientific use. The answers need to be grounded in actual evidence, traceable back to their sources, and transparent enough that a reader can check the reasoning themselves.

2.2 Problem Statement and Motivation

The motivation for this project arises from the growing difficulty of using scientific literature in a reliable and transparent way. Although millions of papers are now available online, researchers still need to manually identify relevant sources, compare results across papers, and verify whether an answer is truly supported by evidence. This process is time-consuming and becomes even harder when a question requires information from several documents.

Several approaches have attempted to tackle this. Retrieval-Augmented Generation (RAG) was an important step forward, combining language models with external document retrieval to reduce hallucinations and improve factual grounding [11]. More recent systems like SQuAI pushed this further by incorporating hybrid search and citation-supported answers over large scientific corpora [12]. That said, most of these approaches share a common limitation: they

retrieve relevant passages and feed them directly to a language model, without constructing any explicit representation of how the evidence pieces relate to one another.

This project addresses three core challenges:

1. Most existing systems rely on abstracts or limited corpora, leaving full-text scientific retrieval largely unexplored at scale.
2. Retrieved passages are rarely structured in a way that explicitly connects claims, citations, and their supporting evidence.
3. There is no arXiv-based dataset specifically designed to evaluate citation-guided evidence expansion and answer faithfulness.

Specifically, we target key deficiencies of existing systems:

- **Limited retrieval scope:** many systems search only abstracts or small collections, which means relevant evidence buried in full-text sections is simply never found.
- **Flat evidence representation:** retrieved passages are treated as isolated chunks, with no structure capturing how claims, citations, and findings relate to each other.
- **Limited traceability:** citations may appear alongside an answer, but the actual path from a generated claim back to its supporting source is rarely made explicit.
- **Weak claim-level verification:** a cited answer can still contain unsupported or overly broad statements if individual claims are not checked against the source text.
- **Difficulty with complex questions:** many scientific questions require pulling evidence from several documents at once, yet most retrieval pipelines have no mechanism for handling this kind of multi-hop reasoning.

Ultimately, the goal of this project is to move beyond simply attaching citations to generated text, toward a system that actually reasons over evidence. To do this, EviGraph-R builds a large-scale retrieval index over the unarXive corpus [6], retrieves relevant chunks through hybrid dense-sparse search, structures the retrieved evidence into a graph, and verifies each extracted claim before producing a final answer. The result is a system designed to generate responses that are not just fluent, but traceable, verifiable, and genuinely grounded in the scientific literature.

3 Objectives

The objectives of this project are focused on three complementary pillars:

- **Build a large-scale hybrid retrieval index:** The unarXive corpus is processed by chunking full-text papers, encoding them as dense and sparse vectors, and storing everything in a vector database enriched with citation and section metadata.
- **Design a multi-agent evidence reasoning pipeline:** A set of specialized agents handles question decomposition, hybrid retrieval, evidence graph construction, claim verification, and citation-aware answer generation.
- **Evaluate the reliability of the system:** A dedicated evaluation framework is built to measure retrieval quality, citation-guided evidence expansion, claim verification, and overall answer faithfulness.

4 Methodology and Project Timeline

This section outlines the methodology followed throughout the project, from the initial problem definition all the way to final delivery, along with a brief overview of the timeline and key milestones.

4.1 Work Methodology

Given the data-intensive and system-oriented nature of this project, working without a structured methodology would have made it difficult to stay focused and consistent across phases. This subsection presents the framework that was chosen to guide the work and explains why it was the most suitable option.

4.1.1 Adapting the CRISP-DM Methodology

CRISP-DM, or Cross-Industry Standard Process for Data Mining, is a structured methodology for data science projects. It includes six phases: business understanding, data understanding, data preparation, modeling, evaluation, and deployment [13]. Together, these phases guide the project from objective definition to final evaluation and delivery.

In this project, the CRISP-DM phases shown in Figure 1.3 were adapted as follows:

- **Business Understanding:** defining the scientific problem, project objectives, and need for reliable question answering over large-scale literature.

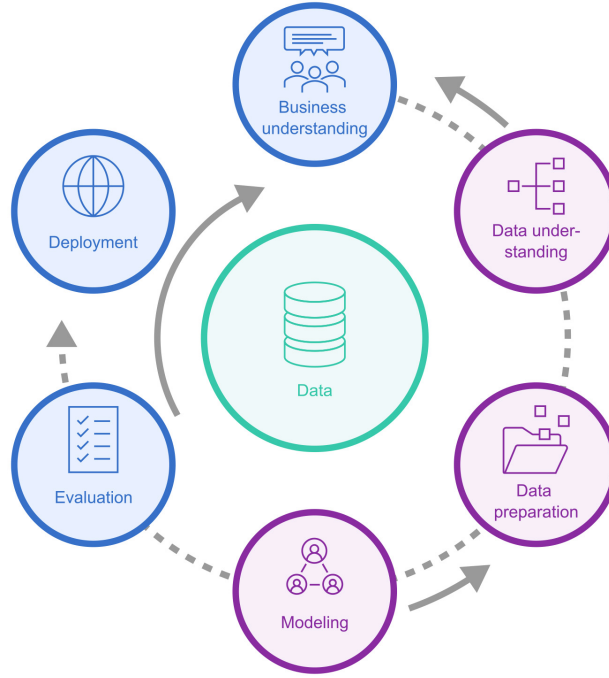


Figure I.3 – The CRISP-DM model (Cross Industry Standard Process for Data Mining) [1].

- **Data Understanding:** analyzing the unarXive corpus, including its structure, meta-data, sections, citations, and limitations.
- **Data Preparation:** preparing the corpus through chunking, dense and sparse embedding generation, citation resolution, and section-title normalization.
- **Modeling:** designing the hybrid retrieval pipeline, multi-agent reasoning workflow, citation-guided evidence graph, and claim verification mechanism.
- **Evaluation:** assessing retrieval quality, citation-guided evidence expansion, claim verification, citation grounding, and answer faithfulness.
- **Deployment:** preparing the system for practical use through a demo interface, installable package, and integration components.

4.1.2 Choice Justification

CRISP-DM was selected because EviGraph-R is both a data-centered research project and a software system. The work required understanding a large scientific corpus, preparing it for retrieval, designing AI-based components, and evaluating answer reliability.

Other methodologies, such as KDD and SEMMA, could also be considered. KDD focuses on discovering useful knowledge from databases through selection, preprocessing, transformation,

data mining, and interpretation [14]. SEMMA, proposed by SAS, focuses on the technical data mining workflow: sampling, exploring, modifying, modeling, and assessing data [15]. CRISP-DM, however, covers a broader lifecycle, including problem understanding, data preparation, modeling, evaluation, and deployment [13].

Previous work has compared these methodologies and shown that their phases can be related to each other. However, CRISP-DM remains broader because it explicitly includes problem understanding and deployment [16]. Table I.1 summarizes this comparison for EviGraph-R.

Tableau I.1 – Brief comparison of KDD, SEMMA, and CRISP-DM.

Criterion	KDD	SEMMA	CRISP-DM
Focus	Knowledge discovery	Data mining	Full project lifecycle
Starting point	Data selection	Data sampling	Problem understanding
Evaluation	Result interpretation	Model assessment	Full evaluation
Deployment	Not central	Not central	Explicit phase
Fit for EviGraph-R	Partial	Partial	Best fit

For these reasons, CRISP-DM was considered the most appropriate methodology. It matched the full lifecycle of EviGraph-R, from understanding the scientific question-answering problem to building the retrieval index, designing the reasoning pipeline, evaluating the system, and preparing the final integration.

4.2 Project Timeline

The project spanned four and a half months, from February 2nd to June 15th, 2026. Figure I.4 illustrates the distribution of work across the main phases of the internship.

Conclusion

In this chapter, we have introduced the host laboratory, defined the project’s context, identified the problems and motivations, outlined the objectives, and discussed the methodologies employed. With these foundational elements established, we are now prepared to understand the theoretical and technical concepts that underpin the development of our project in the next chapter.

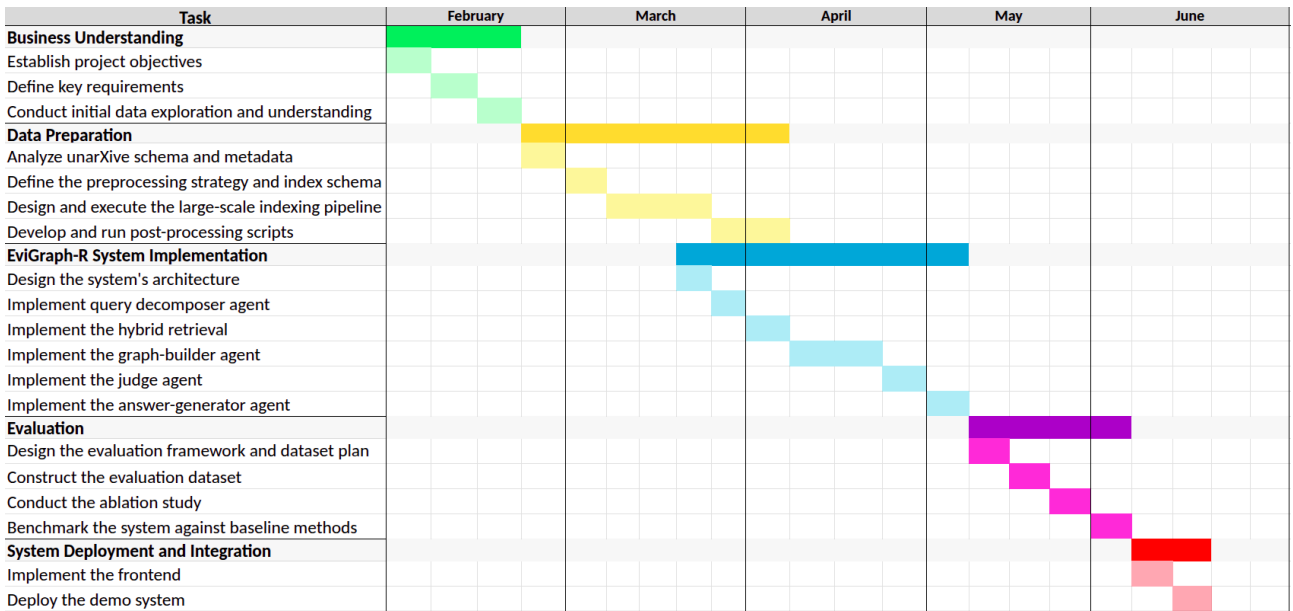


Figure I.4 – GANTT diagram of the project phases and timeline.

Chapter II

Business Understanding and Technical Concepts

Summary

1	Domain-Related Concepts	11
1.1	Structure of Scientific Documents	11
1.2	Scientific Evidence, Claims, and Citations	11
1.3	Citation Intent	11
2	Technical Foundations	12
2.1	Large Language Models (LLMs)	12
2.2	Retrieval-Augmented Generation (RAG)	13
2.3	Natural Language Inference (NLI)	15
2.4	Multi-Agent Systems and Orchestration	15
2.5	Property Graphs for Knowledge Representation	15
2.6	High-Performance Computing (HPC) Infrastructure	16
3	Related Work	17
3.1	Large-Scale Scholarly Datasets	18
3.2	Multi-Agent Retrieval-Augmented Generation	18
3.3	Citation-Grounded Answer Generation	19
3.4	Scientific Question Answering Systems	19

Introduction

In this chapter, we introduce the main concepts behind our system. We first present the structure of scientific documents, as well as the notions of evidence, claims, citations, and citation intent. We then explain the technical foundations of the project, including the methods used to retrieve, structure, and verify scientific evidence. Following that, we review related work that is critical to positioning our approach. Finally, we present the tools and technical environment that support the development of our work.

1 Domain-Related Concepts

To better understand the data and how our system functions, we first need to start by understanding the domain-related concepts.

1.1 Structure of Scientific Documents

Scientific papers usually follow a structured organization, beginning with an **abstract** and often following the **IMRaD** [17] model: **Introduction**, **Methods**, **Results**, and **Discussion**. This structure helps separate the purpose of the study, the approach used, the findings obtained, and their interpretation. In our system, these section boundaries play a practical role. Rather than searching blindly across the full text, the retrieval process can focus on specific sections based on what the question is actually asking for: Methods when the user needs technical details, Results when they are looking for experimental findings.

1.2 Scientific Evidence, Claims, and Citations

Scientific literature is fundamentally built around claims supported by evidence. A **claim** expresses a scientific statement, such as a proposed method, an observed result, or a comparison with prior work. **Evidence** refers to the textual, experimental, or methodological information that supports or challenges that claim. **Citations** connect these claims to previous studies, allowing authors to situate their contribution within the broader scientific record. In our system, these three elements are central: answers must not only retrieve relevant passages, but also identify which claims are supported, trace where the evidence comes from, and understand how citations link different pieces of scientific knowledge together.

1.3 Citation Intent

Not all citations serve the same purpose. An author might cite a previous work to provide background context, to reuse a method, or to compare results. The SciCite dataset formalizes this distinction using three labels: **background**, **method**, and **result comparison** [18]. In our system, classifying citation intent allows us to go beyond treating citations as plain links and instead use them as typed evidence relations within the graph.

2 Technical Foundations

This section covers the core technical concepts that underpin our system, namely large language models, retrieval-augmented generation, natural language inference, multi-agent orchestration, and property graphs for knowledge representation.

2.1 Large Language Models (LLMs)

Large Language Models (LLMs) are deep learning models trained on very large text corpora to understand and generate natural language. Most modern LLMs are based on the Transformer architecture, illustrated in Figure II.1.

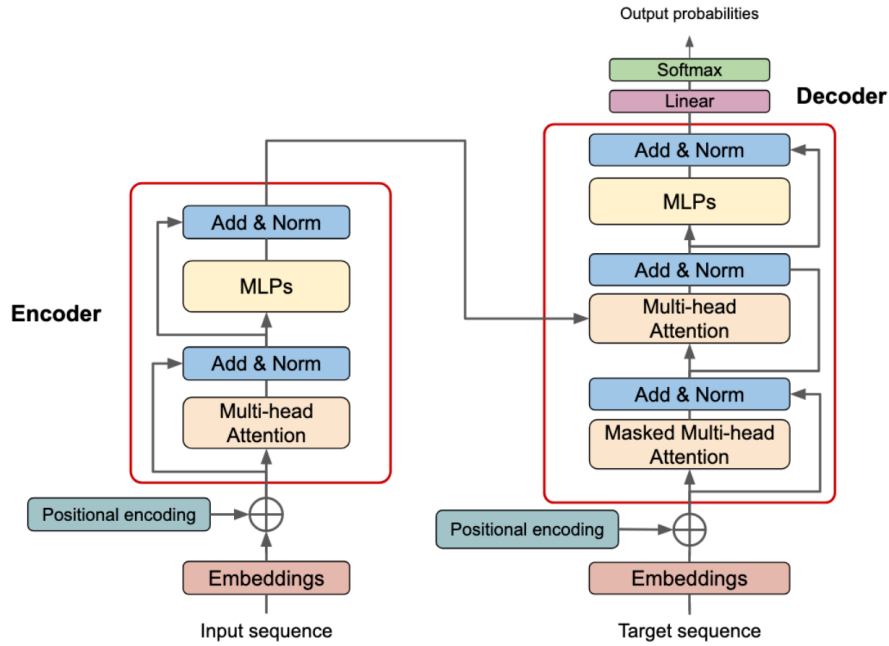


Figure II.1 – The Transformer Model Architecture [2]

This architecture relies on self-attention mechanisms and feed-forward layers to capture contextual relationships between tokens. In the original Transformer model, the encoder is shown on the left and the decoder on the right [2].

Through large-scale pretraining, LLMs learn rich contextual representations that allow them to perform various natural language processing tasks, including text generation, translation, summarization, and question answering. Their ability to generalize across tasks and domains has made them a central component in many recent AI systems [19].

2.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a technique that improves language generation by providing the model with external documents before producing an answer. Instead of relying only on the knowledge stored in the model parameters, a RAG system first retrieves relevant passages from a corpus and then uses them as context for generation [11]. This makes RAG particularly useful for knowledge-intensive tasks such as scientific question answering, where answers must be grounded in reliable sources. As shown in Figure II.2, a RAG pipeline combines a retrieval component with a generative model in order to produce answers grounded in external knowledge sources.

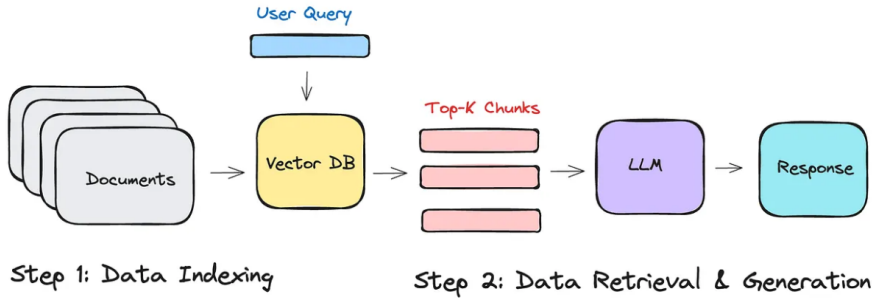


Figure II.2 – Overview of RAG architecture [3].

2.2.1 Lexical Sparse Retrieval

Lexical sparse retrieval represents queries and documents using explicit term-based signals, mainly the words or tokens they contain. Each document is encoded as a sparse vector in which most dimensions are zero, and non-zero dimensions correspond to terms that appear in the text. This type of retrieval is especially effective when the query and the relevant document share exact vocabulary, such as model names, datasets, technical expressions, or domain-specific terminology.

A classical example is BM25, which ranks a document d for a query q by combining term frequency, inverse document frequency, and document-length normalization [20]:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} \quad (\text{II.1})$$

where $f(t, d)$ is the frequency of term t in document d , $|d|$ is the document length, avgdl is the average document length in the corpus, and k_1 and b are tuning parameters.

More recent embedding models, such as BGE-M3, can also produce sparse lexical representations that support term-based matching within neural retrieval pipelines [21].

2.2.2 Semantic Dense Retrieval

Semantic dense retrieval works by mapping both queries and documents into continuous vector embeddings within a shared semantic space. Unlike sparse retrieval, it does not rely on exact word matches but instead tries to capture the underlying meaning of the text. This makes it possible to retrieve relevant passages even when the query and the document phrase things differently but are talking about the same concept.

In practice, dense retrieval relies on neural embedding models that encode both the query and the document into fixed-size vectors. Relevance is then estimated using a similarity function, most commonly cosine similarity:

$$\text{sim}(q, d) = \cos(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} \quad (\text{II.2})$$

where $\mathbf{q}$ and $\mathbf{d}$ are the dense vector representations of the query and the document. The quality of embedding models is often assessed using benchmark suites such as MTEB, which compare models across different retrieval and representation tasks [22].

Models such as BGE-M3 illustrate this direction by supporting dense semantic representations as well as sparse lexical ones [21].

2.2.3 Hybrid Retrieval and Reranking

Hybrid retrieval combines both lexical and semantic matching, taking advantage of what each does best. Sparse retrieval handles precise keyword matching well, while dense retrieval picks up relevant documents that use different wording to express the same idea. A common strategy is to retrieve separate ranked lists from sparse and dense retrievers, then merge them into a single ranking.

Reciprocal Rank Fusion (RRF) is one widely used method for combining ranked lists without requiring score calibration [23]. For a document d , the RRF score is computed as:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \quad (\text{II.3})$$

where R is the set of retrieval runs, $\text{rank}_r(d)$ is the rank of document d in run r , and k is a constant that controls the influence of lower-ranked documents. After fusion, a reranking model can be applied to the top candidates to refine the final order by evaluating the query–document pair more directly.

After fusion, a reranking step can be applied to refine the order of the top retrieved candidates. Unlike dense and sparse retrievers, which usually encode queries and documents separately, a cross-encoder processes the query and candidate passage together and assigns a direct

relevance score.

Since this is more computationally expensive, it is applied only to the top candidates from the first-stage retriever. Models such as `cross-encoder/ms-marco-MiniLM-L-6-v2` are commonly used for this purpose [24].

2.3 Natural Language Inference (NLI)

Natural Language Inference (NLI) is the task of determining the relationship between a premise and a hypothesis, usually classified as **entailment**, **contradiction**, or **neutral** [25]. In our system, NLI serves as the evidence verification step. For each extracted claim, it checks whether the retrieved source actually supports it, contradicts it, or simply does not provide enough information to reach a conclusion. This helps ensure the generated answer stays factually consistent with the retrieved evidence and reduces the chance of unsupported claims making it into the final output.

2.4 Multi-Agent Systems and Orchestration

A multi-agent system brings together several autonomous agents that each handle a specific part of a larger task. Rather than expecting a single model to do everything, the workflow is split across specialized agents responsible for distinct functions such as planning, retrieval, reasoning, verification, and answer generation [26].

This architecture has several practical advantages. It is **flexible**, since individual agents can be updated or replaced without touching the rest of the system. It **scales well** because complex problems can be broken down into smaller, more manageable subtasks. And it tends to **perform better** overall, as agents can pass intermediate results to one another and compensate for each other's limitations [27].

2.5 Property Graphs for Knowledge Representation

A **property graph** (also called a *labeled property graph*, or LPG) is a directed graph where both nodes and edges carry typed labels and an arbitrary set of key-value properties. The model is built on three basic primitives: *nodes* (entities tagged with one or more type labels), *edges* (directed, typed relationships between nodes), and *properties* (key-value pairs attached to either) [4]. One of its main strengths is schema flexibility: new node types and edge labels can be added at any point without restructuring existing data, which makes property graphs a natural fit for domains that evolve over time [28].

Figure II.3 shows a small bibliographic property graph. It contains Author, Article, Conference, and Journal nodes connected by typed edges such as `has_author` and `published_in`.

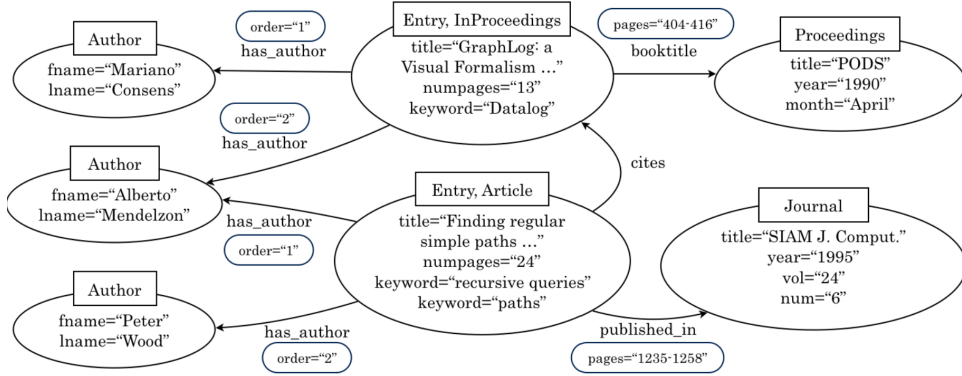


Figure II.3 – Example of a property graph representing bibliographic information [4].

Property graphs differ from RDF-based knowledge graphs in one important way: whereas RDF represents everything, including relationship metadata, as subject–predicate–object triples and relies on formal ontologies for semantics, LPGs allow properties to be attached directly to edges [29]. This makes them more compact and often more intuitive for developers, at the cost of some formal interoperability.[30].

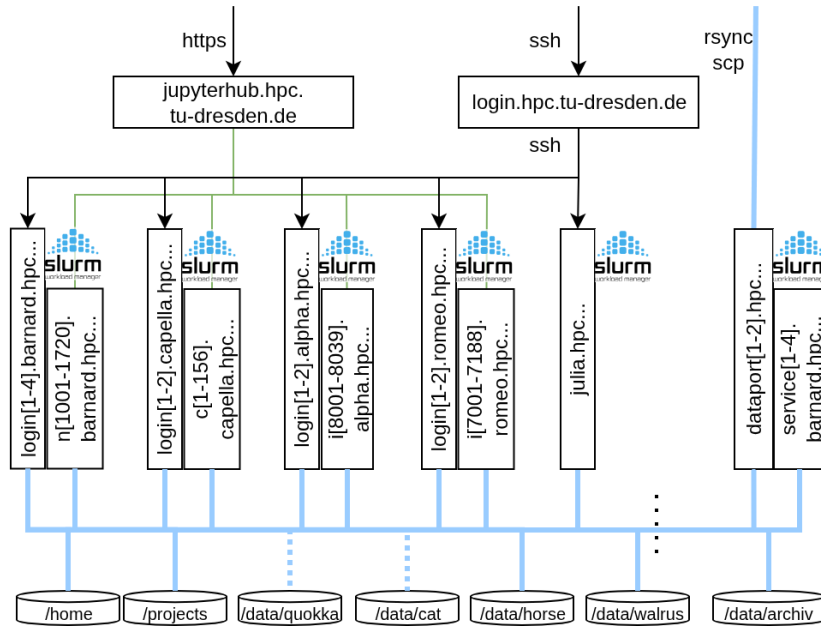
2.6 High-Performance Computing (HPC) Infrastructure

Large-scale scientific NLP workloads, such as embedding millions of text chunks and running experiments over full-text corpora, require computational resources far beyond what a single workstation can provide. This work was carried out on **Capella**, the GPU cluster operated by the Center for Information Services and High Performance Computing (ZIH) at TU Dresden [5].

Capella ranks 3rd among German supercomputers and 51st worldwide in the TOP500 list, while placing 5th on the Green500 list of the most energy-efficient systems globally [31]. It is purpose-built for machine learning and deep learning workloads. Table II.1 summarizes its key hardware specifications, while Figure II.4 provides an architectural overview of the cluster. The WekaIO filesystem (`/data/cat`), exclusive to Capella, is designed specifically for the high I/O throughput demands of AI and ML workflows, making it well-suited to data-intensive pipelines such as large-scale corpus indexing and embedding [32].

Tableau II.1 – Key hardware specifications of the Capella GPU cluster at ZIH, TU Dresden.

Component	Specification
Compute nodes	156 nodes
CPUs	$2 \times$ AMD Genoa per node (32 cores each)
GPUs	$4 \times$ NVIDIA H100 per node (624 total)
GPU memory	94 GB HBM2e per GPU
RAM per node	768 GB DDR5
Peak performance	>38 petaflop/s
Staging storage	2 PB (WekaIO /data/cat), >1 TB/s bandwidth
Cooling	Hot-water cooling with waste-heat recovery


Figure II.4 – Architectural overview of the Capella HPC cluster at ZIH, TU Dresden [5].

3 Related Work

This section reviews existing work across four areas that directly shaped the design of our system: large-scale scholarly datasets, multi-agent RAG architectures, citation-grounded answer generation, and scientific question answering systems.

3.1 Large-Scale Scholarly Datasets

The availability of large-scale, structured scholarly corpora is a prerequisite for scientific question answering at scale. Early datasets such as SciQ [33] and BioASQ [34] were carefully curated but small, covering thousands of questions over narrow biomedical domains. S2ORC [35] significantly raised the bar with 81 million English-language academic papers, yet its LaTeX-parsed full-text subset has since been discontinued, and the remaining PDF-parsed version sacrifices structure and citation fidelity.

UnarXive 2024 [6] addresses these limitations by building directly from arXiv LaTeX sources, preserving document structure, inline citation markers, and mathematical notation. Its 2024 release covers over 2.28 million papers across all scientific disciplines available on arXiv, with an embedded citation network of approximately 40 million references.

Despite the availability of UnarXive, indexing its full text at scale remains an open engineering challenge. A 2.28M-paper corpus, chunked into paragraph-level segments, produces tens of millions of vectors that must be embedded, stored, and queried efficiently. To our knowledge, **no prior work has released a publicly available full-text chunk index** over the complete UnarXive 2024 corpus. The closest related system, SQuAI [12], builds on UnarXive 2024 but restricts retrieval to **abstracts only**, explicitly noting the prohibitive cost of full-text chunking at this scale.

Table II.2 summarises the key characteristics of existing scholarly corpora and positions our index as a contribution in its own right.

Tableau II.2 – Comparison of large-scale scholarly datasets and retrieval indexes. ✓ = available, × = not available, – = not applicable.

Dataset / System	Papers / Size	Full Text	Citation Network	Index Released	Index Scope	Source
SciQ [33]	13.7K Q&A	×	×	–	–	Curated
BioASQ [34]	4K+ Q&A	×	×	–	–	PubMed
S2ORC [35]	81M papers	Partial	✓	×	–	Multi-source
UnarXive 2024 [6]	2.28M papers	✓	✓	×	–	arXiv
SQuAI [12]	2.28M papers	✓	✓	✓	Abstracts only	arXiv
EviGraph-R (ours)	2.28M papers	✓	✓	✓	Full-text chunks	arXiv

3.2 Multi-Agent Retrieval-Augmented Generation

Recent RAG research has increasingly adopted multi-agent architectures to improve retrieval quality and answer faithfulness. Approaches range from leader-worker decomposition [36]

and adversarial robustness training [37] to iterative inner-monologue refinement [38] and task-specialized agents for planning, verification, and generation [39, 40]. Closer to our setting, MAIN-RAG [41] introduced training-free document filtering and evidence scoring, later extended by RAGENTA [42] with an answer revision agent and inline source attribution. SQUAI brought this paradigm to the scientific domain, building a four-agent pipeline over the full arXiv corpus with hybrid retrieval and per-claim citations [12].

Our work builds upon these foundations in multi-agent orchestration and citation-grounded generation, but further advances the state of the art by structuring retrieved evidence as a property graph rather than a flat collection of passages, enabling explicit reasoning over relational chains of scientific claims.

3.3 Citation-Grounded Answer Generation

Grounding generated text in verifiable sources has become a central concern in LLM research. Early work introduced the *Attributable to Identified Sources* (AIS) framework [43] as a human evaluation protocol for faithfulness, later extended through automated NLI-based metrics [44]. ALCE [45], one of the first large-scale benchmarks for attributed generation, formalized citation recall and precision at the sentence level. Since then, two main paradigms have emerged: *end-to-end* methods that generate answers and citations jointly [45], and *post-hoc* approaches that assign citations after generation through NLI-based verification [46]. In the scientific domain, RAGENTA [42] and SQUAI [12] integrated inline citations into multi-agent pipelines over arXiv.

Despite these advances, most scientific QA systems still fall short when it comes to attribution granularity. Citations tend to be attached at the document or abstract level, the reasoning behind an answer is rarely visible, and the evidence structure is never exposed to the user. EVIGRAPH-R addresses these gaps by grounding each answer sentence in specific chunks and sections, verifying extracted claims through both NLI and LLM-based judgment, and presenting a transparent evidence graph that makes the full reasoning path traceable and independently checkable.

3.4 Scientific Question Answering Systems

Scientific QA tasks span a wide range of input sources and answer formats. Document-grounded settings such as Qasper [47] derive answers from a provided paper, while open-domain settings such as LitSearch [48] and SciQA [49] require retrieval from large corpora. Answer formats range from multiple-choice questions [50] to short factoid responses in biomedical tasks [51, 52] and long-form free-text answers [49].

Dedicated systems have emerged to tackle these settings. PaperQA2 [53] performs agentic RAG over user-supplied PDFs with citation support: it orchestrates a multi-step pipeline of retrieval, re-ranking, and answer synthesis driven by an LLM agent. However, it operates on small, user-defined corpora rather than a large pre-indexed archive, and its evidence graph remains implicit within the agent’s context window rather than being exposed to the user. BioRAGent [54] adopts a similarly agentic retrieval strategy for biomedical QA, combining dense passage retrieval with an LLM-driven reasoning loop to produce inline citations, yet it is restricted to the biomedical domain and does not construct an explicit, inspectable evidence structure. Commercial tools such as Elicit, Consensus, and SciSpace offer literature search and synthesis at scale, but are closed-source, do not expose their reasoning, and provide only document-level attribution. SQuAI [12] is the most comparable open system: it covers 2.28M arXiv papers with inline citations, yet retrieval is abstract-only and the evidence structure is not exposed to the user.

In contrast, EVIGRAPH-R takes an agentic approach at open-domain scale, but rather than keeping evidence implicit in the agent’s context, it structures it into an explicit graph exposed directly to the user, enabling verifiable chunk-level attribution across 2.28M full-text papers.

Table II.3 summarises these systems and positions EVIGRAPH-R among them.

Tableau II.3 – Comparison of scientific QA systems across key dimensions. ✓ = yes, × = no, ~ = partial.

System	Open-domain	Full-text retrieval	Inline citations	Transparent evidence	Open source
ScienceQA [50]	×	×	×	×	✓
BioRAGent [54]	×	~	✓	×	✓
PaperQA2 [53]	×	✓	✓	×	✓
Elicit [55]	✓	×	×	×	×
Consensus [56]	✓	×	×	×	×
SciSpace [57]	✓	×	~	×	×
SQuAI [12]	✓	×	✓	×	✓
EviGraph-R (ours)	✓	✓	✓	✓	✓

EVIGRAPH-R is, to our knowledge, the first open-domain scientific QA system to combine full-text chunk-level retrieval at the scale of 2.28M papers, reasoning over a structured evidence graph, per-sentence inline citations grounded in specific sections, and full transparency of the reasoning chain to the user.

Conclusion

In this chapter, we have outlined the key domain-related concepts, explored the main technical foundations, and reviewed related work positioning our work within scientific question answering systems. We also presented the technical environment used to support large-scale indexing and experimentation. With a solid understanding of these foundational elements, we are now prepared to proceed to the next stage: data understanding and preparation.

Chapter III

Data Understanding and Preparation

Summary

1	Dataset Source	23
1.1	The UnarXive 2024 Corpus	23
1.2	Raw Data Format and Schema	24
1.3	Dataset Limitations and Quality Issues	25
2	Index Creation Pipeline	25
2.1	Paper Normalization and Preprocessing	25
2.2	Chunking and Embedding Generation	26
2.3	Vector Database Ingestion on HPC	27
2.4	Post-Indexing Enrichment	28
2.5	End-to-End Pipeline Overview	29
3	Exploratory Analysis	30
3.1	Index Schema and Payload Structure	30
3.2	Index Statistics and Coverage	31
3.3	Temporal Coverage of the Index	32
3.4	IMRaD Structural Label Distribution	32

Introduction

This chapter outlines the essential steps taken to prepare the data for the system, covering four complementary aspects: the unarXive 2024 corpus that serves as our knowledge base, the distributed indexing pipeline designed to process millions of papers at scale, the resulting vector index enriched with citation and section metadata, and an exploratory analysis of the corpus structure and its inherent quality challenges.

1 Dataset Source

This section describes the data source used in EviGraph-R, covering where it comes from, how it is structured, and the quality issues that arise when working with it at scale.

1.1 The UnarXive 2024 Corpus

UnarXive 2024 [6] is a large-scale open scientific corpus built directly from arXiv L^AT_EX sources, covering over 2.28 million full-text papers across all disciplines on arXiv, from physics and mathematics to computer science, quantitative biology, and economics.

What sets it apart from PDF-parsed alternatives like S2ORC [35] is that it preserves the original document structure, including section boundaries, inline citation markers of the form `cite:REFID`, mathematical formulas rendered as `[FORMULA]` placeholders, and a bibliography dictionary mapping each reference to its metadata.

The corpus covers papers submitted between 1991 and 2024 and is distributed as JSONL files through HuggingFace Datasets, making it straightforward to integrate into large-scale processing pipelines. As shown in Figure III.1, submissions grew steadily over two decades, with a sharp acceleration after 2018 driven largely by Computer Science, reaching close to 200,000 papers in 2024 alone.

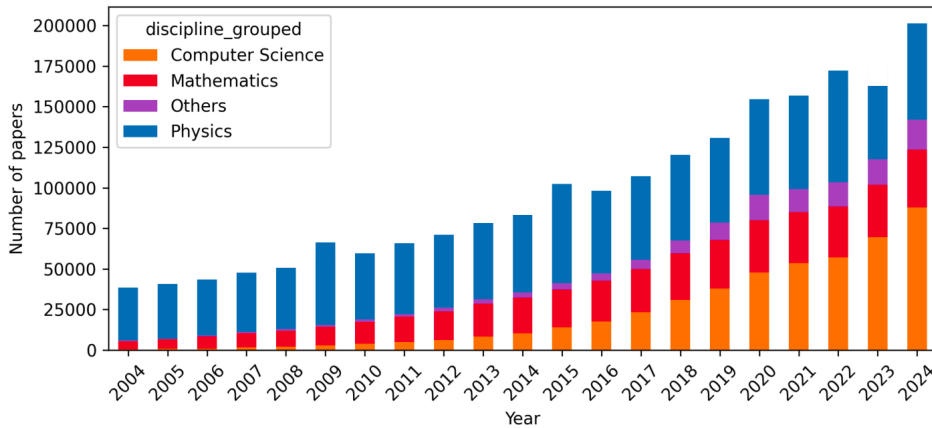


Figure III.1 – Annual number of publications in unarXive 2024, broken down by discipline. [6].

These properties make it the most suitable foundation for a full-text chunk-level retrieval index at the scale required by EviGraph-R.

1.2 Raw Data Format and Schema

Each paper is stored as a self-contained JSON object whose top-level fields include a unique arXiv identifier (`paper_id`), an optional DOI, a cleaned abstract, a bibliography dictionary (`bib_entries`) mapping reference identifiers to their DOI, arXiv ID, or OpenAlex ID, and a sections dictionary carrying the raw text and inline citation markers of the form `{{cite:REFID}}` for each section. This structure allows the preprocessing pipeline to extract citation spans with precise character offsets and maintain full traceability between indexed chunks and their source papers, as illustrated in Listing III.1.

Listing III.1 – unarXive 2024 paper representation schema

```

1 { "paper_id": "2412.00056",
2   "metadata": {
3     "id": "2412.00056",
4     "authors": "...",
5     "title": "Improving Medical ...",
6     "doi": "..",
7     "categories": "cs.CV cs.AI",
8     "discipline": "Computer Science",
9     "citation_count": 5
10  },
11  "abstract": "In recent years, vision-language...",
12  "bib_entries": { "...": { "...": "..." } },
13  "ref_entries": { "...": { "...": "..." } },
14  "sections": {
15    "Introduction": {
16      "text": "In recent years...",
17      "cite_spans": [
18        { "start": 42,
19          "end": 52,
20          "ref_id": "abc123",
21          "text": "{{cite:abc123}}" }
22      ],
23      "ref_spans": [ "..." ]
24    }
25  }
26 }
```

1.3 Dataset Limitations and Quality Issues

Despite its richness, the unarXive 2024 corpus presents three categories of quality challenges that must be addressed before it can serve as a reliable retrieval index.

- **Section structure noise:** Only **23%** of section titles conform to standard scientific conventions (IMRaD), and about **9.7%** of papers contain only missing or uninterpretable section titles, making section-aware retrieval unreliable without prior normalization.
- **Citation resolution at scale:** **48.8%** of bibliography entries lack any external identifier (DOI, arXiv ID, or OpenAlex ID), and the full corpus contains approximately **40 million** citation references that require calls to external APIs to resolve.
- **No ready-to-use retrieval index:** unarXive 2024 is distributed as a raw JSONL dataset, not a searchable index. The closest existing system, SQuAI [12], built a FAISS index over this corpus but restricted retrieval to abstracts only, discarding the rich full-text content. This limitation directly motivates the construction of a new chunk-level hybrid index over the complete full-text corpus.

2 Index Creation Pipeline

This section details the **four-stage pipeline** designed to transform the raw unarXive 2024 corpus into a searchable hybrid retrieval index. Each stage is presented in order, from paper normalization and preprocessing, through chunking and embedding, to vector database ingestion and post-indexing enrichment.

2.1 Paper Normalization and Preprocessing

Before any chunking or embedding, each raw paper passes through a normalization stage that ensures clean, structured input for the downstream pipeline. This stage handles four main operations.

Section filtering. Structurally uninformative sections are excluded from indexing entirely. The skip list covers acknowledgements, references, bibliography, table of contents, and lists of figures or tables.

Citation marker processing. In unarXive, inline citations appear as `{{cite:<ref.id>}}` placeholders embedded directly in the section text. The preprocessor removes these markers to produce clean readable prose, while recording the exact position of each citation in the text. Each chunk therefore carries a `cite_spans` field that maps every citation to its location and

its bibliographic metadata (DOI, arXiv ID, OpenAlex ID), which the multi-agent system later uses to expand evidence across cited papers.

Figure and table reference cleaning. Inline markers of the form `{{figure:<uuid>}}` and `{{table:<uuid>}}` are replaced with human-readable strings `[Figure: <caption>]` or `[Table: <caption>]` when a caption exists, and removed otherwise.

Metadata normalization. Author lists are parsed from structured name arrays with a fallback to raw string splitting. DOI strings are normalized to bare form by stripping URL prefixes. The publication year is extracted from the paper’s versions list with a fallback to `update_date`.

2.2 Chunking and Embedding Generation

To make the corpus suitable for efficient retrieval, the raw scientific text must first be transformed into fixed-size, semantically coherent vector units; for this purpose, several chunking and embedding strategies were implemented and compared.

Chunking strategy : Token counting uses the BAAI/bge-m3 tokenizer and all splits are forced to occur at sentence boundaries. Two strategies are applied depending on the content type. **Abstracts** are treated as a single logical unit and stored as one chunk when their token count is at or below 500 tokens. When an abstract exceeds this threshold, a sliding window is applied with a 50-token overlap. **Body sections** use a wider sliding window of 900 tokens with a 30-token overlap, producing overlapping chunks that preserve local context across section boundaries. Each chunk receives a deterministic identifier computed as:

$$\text{chunk_uid} = \text{SHA-1}(\text{paper_id} + \text{section_label} + \text{chunk_index} + \text{text}) \quad (\text{III.1})$$

This ensures reproducible identifiers across re-runs and enables idempotent upserts into the vector database. The production run produced **69,026,381 total chunks** from 2,284,380 papers, averaging approximately **30.22 chunks per paper**.

Embedding model : Each chunk is encoded using BAAI/bge-m3 [21], a state-of-the-art multilingual embedding model that produces both a dense vector and a sparse lexical representation from the same forward pass, enabling true hybrid retrieval without running two separate models. Table III.1 summarises its key parameters.

Tableau III.1 – BAAI/bge-m3 embedding model specifications.

Parameter	Value
Dense vector dimension	1,024
Sparse representation	Yes (SPLADE-style lexical weights)
Max sequence length	8,192 tokens
Inference batch size	512 sequences per GPU call
Data type	float16
Normalization	L2 (required for Cosine distance)

HPC execution : The chunking and embedding phase is distributed across **200 parallel SLURM array tasks** on the Capella GPU cluster, each allocated 1 GPU, 8 CPU cores, and 170 GB of RAM with a 24-hour wall-time limit. Batch files are distributed round-robin: task i processes batches at indices $\{i, i+200, i+400, \dots\}$, giving each task approximately 11 batches (11,000 papers). After processing each batch, the task writes a shard `.jsonl` file and a `.done` sentinel to disk. On restart with the `--resume` flag, any shard whose sentinel exists is skipped, making the phase fully idempotent.

2.3 Vector Database Ingestion on HPC

Once all shard files are complete, the ingestion phase begins. A dedicated Capella node runs a self-hosted **Qdrant** instance inside a **Singularity** container [58], acting as a centralized vector database server. Unlike Docker, Singularity is designed for HPC environments and does not require root privileges, making it the standard containerization solution on clusters such as Capella [32]. Multiple worker nodes simultaneously stream their completed shards into this single Qdrant instance over the cluster’s high-speed InfiniBand interconnect, allowing embedding computation and database ingestion to overlap in time and significantly reducing the total pipeline duration.

Collection configuration. Table III.2 summarises the Qdrant collection parameters used for the production index.

Tableau III.2 – Qdrant collection configuration for `unarxiv_chunks`.

Parameter	Value
Dense vector dimension	1,024 (Cosine metric)
Sparse vector	BGE-M3 SPLADE-style lexical weights
Quantization	int8 scalar ($\approx 4\times$ memory reduction)
Vector storage	Memory-mapped (mmap)
HNSW m	32
HNSW $ef_construct$	128
HNSW ef	64
Indexed payload fields	10 (<code>id_arxiv</code> , <code>section_title</code> , <code>year</code> , <code>categories</code> , and 6 others)
BM25 full-text index	Built over <code>embed_text</code> field

Fault-tolerant ingestion. Each shard is upserted in batches of 256 points with exponential backoff retry on timeout errors (up to 5 attempts, from 2s to 60s). After each successful shard, a record is appended to an `ingested_shards.jsonl` progress log so that any interrupted job resumes exactly where it left off without duplicating work. A Qdrant collection snapshot is created every 50 ingested shards for disaster recovery. The full ingestion of **69,026,381 points** across 2,215 shards completed in approximately **1.5 weeks**.

2.4 Post-Indexing Enrichment

A key architectural decision is to decouple enrichment from ingestion entirely. Resolving 40 million citation references and classifying section titles inline during embedding would have blocked GPU compute on network I/O and is estimated to have added days of pipeline delay. Instead, two post-processing scripts update Qdrant payload fields in-place after the index is live.

Script 1 : Citation identifier resolution. For chunks whose bibliography entries lack DOIs or arXiv IDs, the resolver queries the **OpenAlex** and **arXiv** APIs using 16 concurrent coroutines at 1 request/second per host. Resolved identifiers are written back to the `cite_spans` payload field of affected chunks. The full run over 40 million citation references completed in approximately **43 hours**.

Script 2: IMRaD Section Normalisation. To enable section-aware retrieval, each chunk must be labeled with its IMRaD category. We fine-tuned a **DistilBERT**-based sequence classifier [59] on the `saier/unarXive-imrad-clf` dataset, a labeled collection of arXiv paper sections covering five categories: Introduction, Methods, Results, Discussion, and Other. The

model is released publicly on HuggingFace as `lostelf/section-classifier-imrad`¹ and achieves an F1 score of **0.776** and an accuracy of **77.1%** on the evaluation set.

Classification is applied via a two-tier strategy: a curated set of regular expression patterns handles the majority of standard section titles (e.g., `^intro(duction)?$, ^methods?$`), while the fine-tuned classifier resolves ambiguous cases using GPU batch inference. The resulting `imrad_label` is written back to each chunk’s payload.

Since section labels serve as soft contextual signals for ranking rather than hard filters, occasional misclassifications have limited downstream impact: a mislabeled chunk remains retrievable on semantic grounds.

Table III.3 reports the label distribution over the full index, with IMRaD coverage reaching **94.6%** of all indexed chunks.

Tableau III.3 – IMRaD label distribution after post-indexing normalisation.

Label	Chunk count	Share of total
Results	30,336,916	43.9%
Introduction	13,991,399	20.3%
Methods	13,725,554	19.9%
Discussion	7,230,836	10.5%
Non-IMRaD (skipped)	2,805,995	4.1%
Unlabeled	935,681	1.4%
Total	69,026,381	100%

2.5 End-to-End Pipeline Overview

Figure III.2 summarises the complete index creation pipeline. The four stages : preprocessing, chunking and embedding, vector database ingestion, and post-indexing enrichment, are designed to be independently resumable and fault-tolerant, enabling the pipeline to recover from HPC job interruptions without data loss or duplicated work. The final output is the `unarchive_chunks` Qdrant collection, which serves as the retrieval backbone of the EviGraph-R system described in the next chapter.

¹<https://huggingface.co/lostelf/section-classifier-imrad>

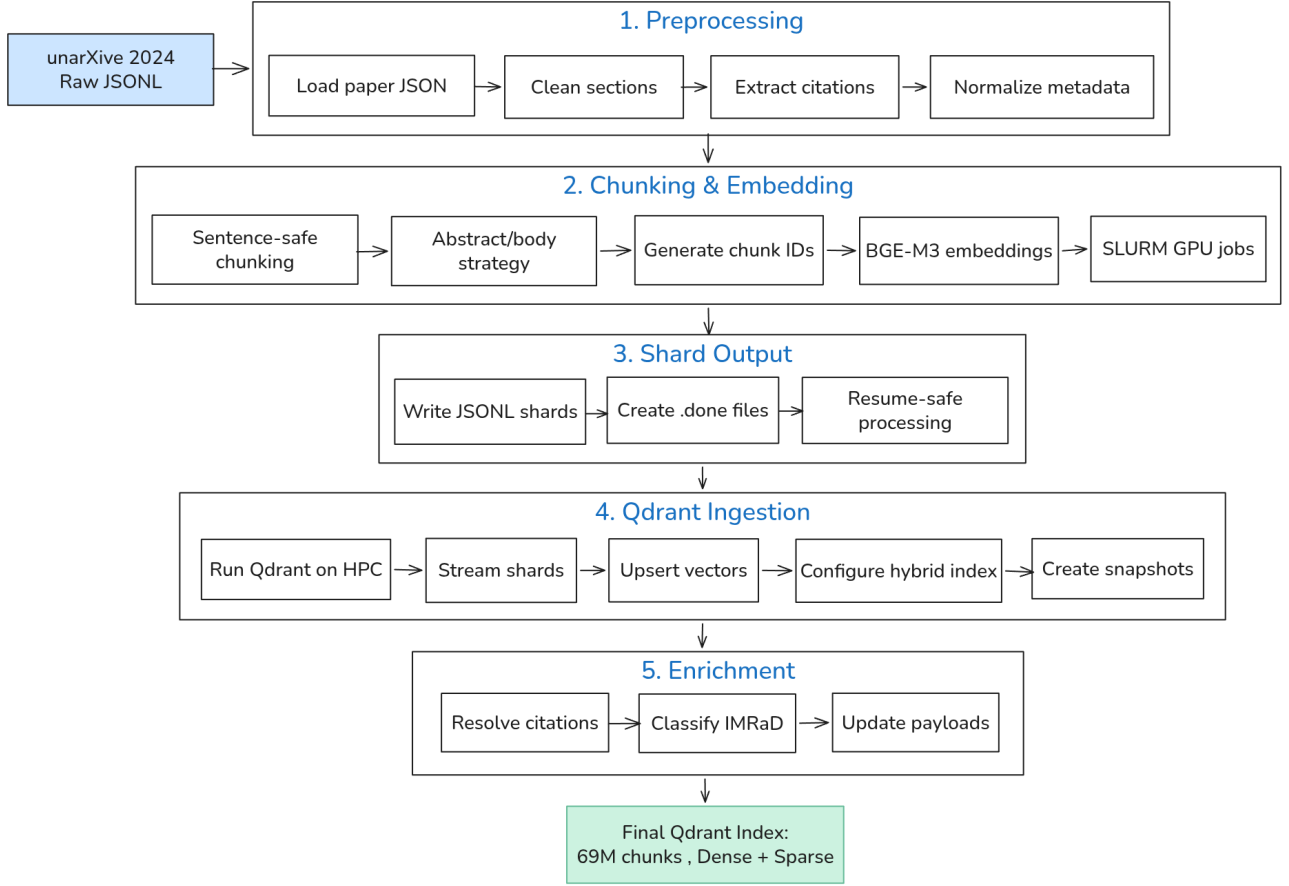


Figure III.2 – End-to-end overview of the EviGraph-R index creation pipeline.

3 Exploratory Analysis

This section characterises the final production index of EviGraph-R: its payload schema and vector configuration, aggregate collection statistics, and a corpus-level analysis of temporal coverage, structural annotation, and retrieval performance.

3.1 Index Schema and Payload Structure

Each point stored in the Qdrant collection `unarxive_chunks` carries two vector representations and a rich metadata payload. The dense vector encodes the semantic content of the chunk as a 1,024-dimensional float16 embedding, while the sparse vector provides SPLADE-style lexical weights for keyword matching. Together, they enable the hybrid retrieval strategy used by EviGraph-R. Table III.4 describes the full payload schema attached to each point.

Tableau III.4 – Payload schema of the `unarxive_chunks` Qdrant collection.

Field	Type	Description
<code>chunk_uid</code>	string	SHA-1 deterministic identifier
<code>paper_id_arxiv</code>	string	arXiv paper identifier
<code>paper_doi</code>	string	Normalized DOI (bare form)
<code>title</code>	string	Paper title
<code>authors</code>	list	Parsed author names
<code>categories</code>	list	arXiv subject categories
<code>year</code>	integer	Publication year (nullable)
<code>chunk_type</code>	string	abstract or subsection
<code>chunk_index</code>	integer	Position of chunk within paper
<code>total_chunks</code>	integer	Total chunks for this paper
<code>section_title</code>	string	Original section heading
<code>embed_text</code>	string	Text used for embedding
<code>cite_spans</code>	list	Citation spans with offsets, DOI, arXiv ID, OpenAlex ID
<code>imrad_label</code>	string	IMRaD category

3.2 Index Statistics and Coverage

Table III.5 summarises the key quantitative characteristics of the final production index, reflecting the output of the full pipeline.

Tableau III.5 – Final index statistics for `unarxive_chunks`.

Metric	Value
Total papers indexed	2,284,380
Total chunks (Qdrant points)	69,026,381
Average chunks per paper	30.22
Dense vector dimension	1,024
Sparse vectors	Yes (SPLADE-style)
BM25 full-text index	Yes (over <code>embed_text</code>)
Quantization	int8 scalar
IMRaD labeling coverage	94.6%
Citation identifier coverage	~71.2% (post-resolution)
Indexed payload fields	10

EviGraph-R is the first system to provide a full-text chunk-level hybrid index over the unarXive corpus, combining dense semantic vectors, sparse lexical weights, and a BM25 keyword

index within a single Qdrant collection, enriched with citation metadata and IMRaD section labels.

3.3 Temporal Coverage of the Index

Figure III.3 shows the distribution of indexed chunks across publication years from 2015 to 2024. The corpus grows steadily throughout the decade, with a marked acceleration beginning in 2020 reflecting the surge in arXiv submissions during and after the COVID-19 pandemic. The 2024 cohort is the largest single year at 5.8 million chunks, though the index captures only papers available in the unarXive snapshot and therefore under-represents the most recent submissions. Collectively, the ten years shown account for the bulk of the 69 million indexed chunks, confirming broad temporal coverage across modern scientific literature.

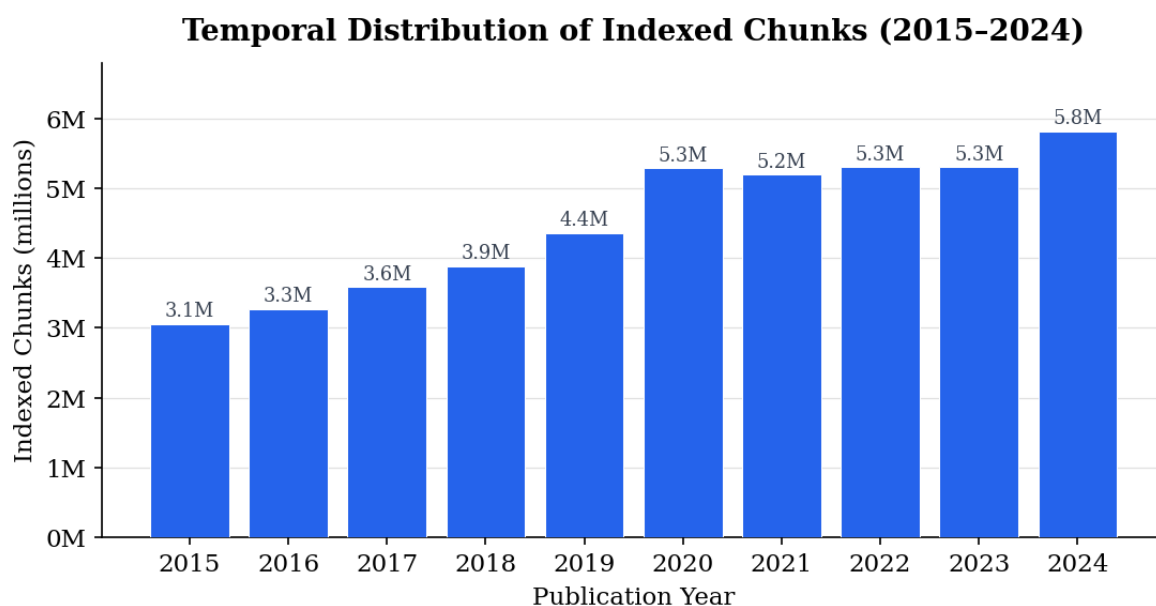


Figure III.3 – Number of indexed chunks per publication year

3.4 IMRaD Structural Label Distribution

Each chunk in the index carries an IMRaD label (*Introduction*, *Methods*, *Results*, or *Discussion*) assigned by a fine-tuned classifier, with heuristic and sequence-repair fallbacks. Figure III.4 shows the resulting distribution across the full collection.

Results sections are the most prevalent category (30.3 million chunks, 43.9%), consistent with the mathematical and physics-heavy nature of the unarXive corpus where derivation and proof sections are annotated as Results. *Introduction* and *Methods* sections each account for

roughly 20% of labeled chunks. Crucially, only 1.4% of chunks remain unlabeled after the full labeling pipeline, and a further 4.1% are explicitly flagged as non-IMRaD (e.g. front-matter, acknowledgements) and excluded from IMRaD-filtered retrieval paths. The overall labeled coverage of 94.6% confirms that the structural annotation is nearly complete at corpus scale.

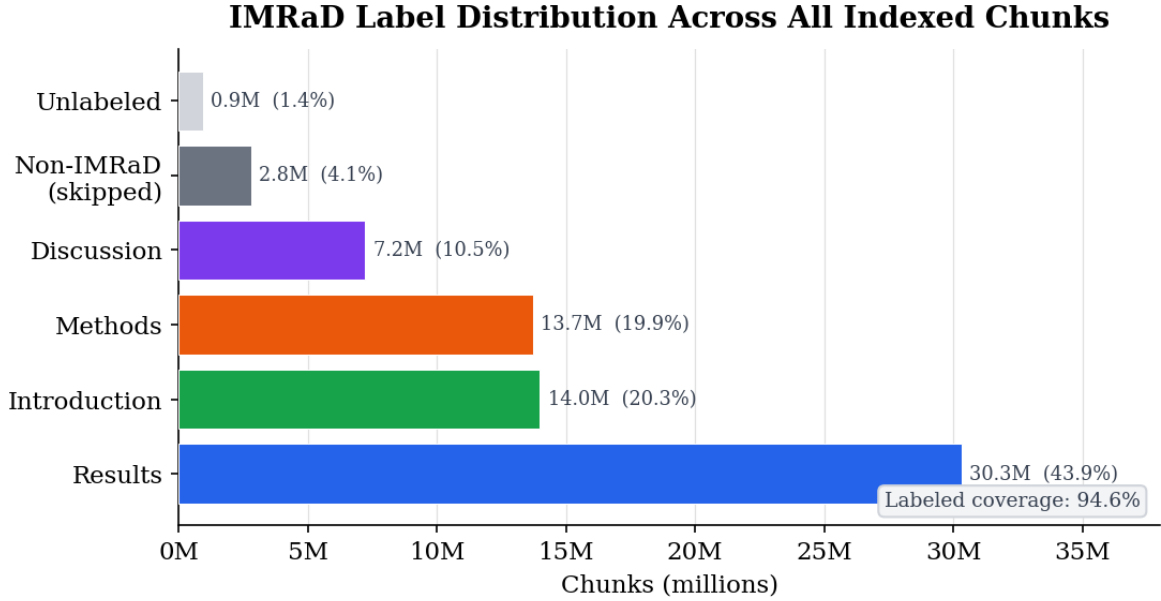


Figure III.4 – IMRaD label distribution across all 69 million indexed chunks after post-processing

Conclusion

This chapter established the data foundation of EviGraph-R, from the characterization of the unarXive 2024 corpus and its quality challenges, through the distributed indexing pipeline on the Capella HPC cluster, to the final 69-million-chunk hybrid retrieval index enriched with citation metadata and IMRaD labels. With this foundation in place, the next chapter presents the system architecture and multi-agent reasoning pipeline built on top of it.

Chapter IV

System Design and Architecture

Summary

1	Overview of EviGraph-R Agentic Architecture	35
1.1	High-Level Architecture	35
1.2	Orchestration, Structured Reasoning, and Observability	35
2	Hybrid Retrieval Node Design	36
2.1	BGE-M3 Sparse Vector Search	37
2.2	BGE-M3 Dense Vector Search	37
2.3	Reciprocal Rank Fusion (RRF)	38
2.4	Cross-Encoder Reranking	38
2.5	Section Filtering and Budget-Weighted Scoring	38
3	Agent Roles and Responsibilities	39
3.1	Query Decomposer Agent	39
3.2	Graph Builder Agent	40
3.3	Judge Agent	43
3.4	Answer Generator Agent	43
4	End-to-End Walkthrough	45
4.1	Query Decomposition	45
4.2	Sub-query Retrieval	46
4.3	Evidence Graph Construction	47
4.4	Claim Verification	47
4.5	Answer Generation and Citations	49

Introduction

This chapter presents the system design and architecture of EviGraph-R. We first introduce the overall structure of the system and the key frameworks on which it is built, before examining in detail each component, from the hybrid retrieval node to the role of each of the four agents. Finally, a concrete end-to-end walkthrough illustrates how an incoming scientific query traverses the full pipeline, from decomposition through evidence graph construction to a citation-grounded answer.

1 Overview of EviGraph-R Agentic Architecture

EviGraph-R’s multi-agent query pipeline processes incoming scientific queries through a sequence of four specialized agents, each building on the outputs of the previous. The following sections describe the high-level structure of this pipeline and the key technologies underpinning its implementation.

1.1 High-Level Architecture

EviGraph-R’s query pipeline is structured as a sequential chain of five specialized components, each responsible for a distinct stage of the question answering process. As illustrated in Figure IV.1, an incoming user query is first passed to the **Decomposer Agent**, which analyzes the query intent, breaks it into targeted sub-queries, and assigns a retrieval budget to each. These sub-queries are then forwarded to the **Retrieval Node**, which embeds them, searches the Qdrant index, and returns the top- k most relevant chunks. The retrieved chunks are handed to the **Graph Builder Agent**, which extracts atomic scientific claims, establishes evidential relations between them, and constructs an Evidence Property Graph. This graph is then passed to the **Judge Agent**, which verifies each claim and assigns verdicts along with rejection reasons where applicable. Finally, the **Answer Agent** synthesizes a fluent, citation-grounded response from the supported claims, returning a final answer together with evidence traces back to the source literature.

1.2 Orchestration, Structured Reasoning, and Observability

Beyond the sequence of agents, EviGraph-R relies on an execution layer that coordinates the pipeline and keeps its intermediate outputs structured, traceable, and easy to debug.

LangGraph is used to orchestrate the full query pipeline as a stateful graph. Each node represents one step, such as decomposition, retrieval, graph construction, judgment, or answer

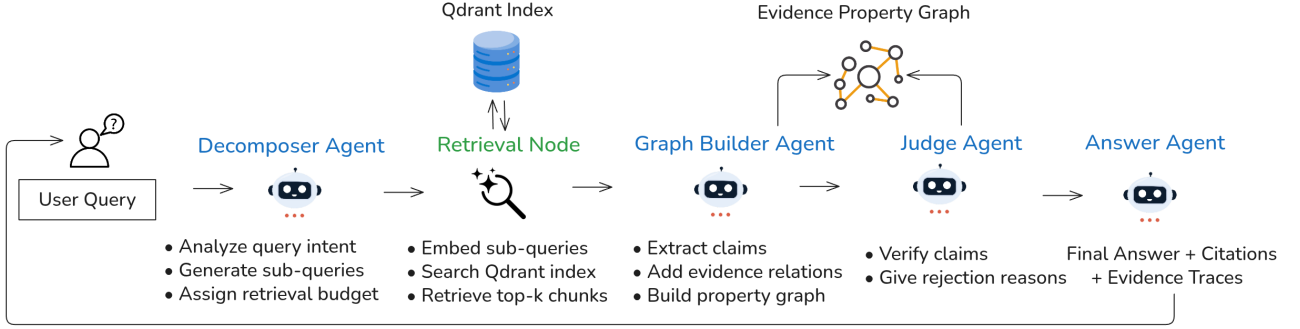


Figure IV.1 – High-level architecture of the EviGraph-R multi-agent system.

generation. The shared state is updated at each stage and passed forward, making the execution flow explicit and allowing conditional behavior, such as stopping when a fatal error occurs [60].

Pydantic defines the structured objects exchanged between agents, including sub-queries, retrieved evidence, extracted claims, verification verdicts, and final answers. This avoids unstructured intermediate data and makes validation, debugging, and extension easier [61].

DSPy is used to express the LLM-based reasoning modules, such as query decomposition, claim extraction, and evidence reasoning. It provides a declarative way to define prompts and structured outputs, while also supporting later prompt and demonstration optimization [62].

LangSmith provides observability by recording pipeline traces, including LLM calls, intermediate outputs, node transitions, and errors. This helps identify whether a failure comes from decomposition, retrieval, claim extraction, verification, or answer generation [63].

Together, LangGraph controls the workflow, Pydantic structures the exchanged data, DSPy defines the reasoning modules, and LangSmith makes the execution trace observable.

2 Hybrid Retrieval Node Design

The Hybrid Retrieval Node takes each sub-query produced by the Decomposer and turns it into a ranked list of relevant document chunks. It does this through a sequence of steps: sparse and dense search results are first merged using Reciprocal Rank Fusion, then refined by a cross-encoder reranker, and finally adjusted by a section-aware score boost.

2.1 BGE-M3 Sparse Vector Search

Sparse retrieval works by matching queries to documents based on shared vocabulary, giving higher scores to chunks that contain the same or similar terms as the query. In EviGraph-R, this is handled through two mechanisms depending on which embedding mode is active.

BGE-M3 lexical weights. We opted for the BGE-M3 model [21] where sparse representations are produced natively by the model in a single forward pass alongside the dense vector. BGE-M3 computes a *lexical weight* w_t for each token t present in the text, yielding a sparse vector

$$\mathbf{s} = \{t \mapsto w_t \mid t \in V, w_t > 0\} \quad (\text{IV.1})$$

where V is the model vocabulary. These weights capture term importance in a data-driven way, going beyond simple term frequency. Both query and passage sparse vectors are stored in Qdrant under the `sparse` named-vector field, and inner-product matching is used at search time.

2.2 BGE-M3 Dense Vector Search

Dense retrieval represents both queries and document chunks as continuous vectors in a shared semantic space, enabling matching on meaning rather than surface form. EviGraph-R uses **BGE-M3** [21] (BAAI/bge-m3) as its default embedding model. BGE-M3 maps text to 1024-dimensional vectors and supports context windows of up to 8192 tokens, making it well-suited for scientific passages that may span several paragraphs.

All embeddings are L2-normalised before storage, so the similarity between a query vector $\mathbf{q}$ and a chunk vector $\mathbf{d}$ reduces to the cosine similarity:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \mathbf{q}^\top \mathbf{d}, \quad \|\mathbf{q}\| = \|\mathbf{d}\| = 1 \quad (\text{IV.2})$$

Dense vectors are stored in Qdrant under the `dense` named-vector field and indexed with HNSW [64] (with $M = 32$, $ef_construct = 128$ on HPC) for approximate nearest-neighbour search. At query time, the top- K ($K = 100$) nearest chunks are retrieved from the HNSW index for each sub-query.

2.3 Reciprocal Rank Fusion (RRF)

Sparse and dense retrieval return separate ranked lists that are not directly score-comparable. Reciprocal Rank Fusion (RRF) [23] merges them into a single unified ranking without requiring score normalisation. Given n ranked lists $\{R_1, \dots, R_n\}$ and a constant k , the RRF score of a document d is:

$$\text{RRF}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)} \quad (\text{IV.3})$$

where $\text{rank}_i(d)$ is the position of d in list R_i (documents absent from a list are assigned a large rank). Our system uses Qdrant’s native `FusionQuery(fusion=Fusion.RRF)` operator with the standard constant $k = 60$. Each prefetch step, dense and sparse, retrieves up to 100 candidate chunks, and RRF fusion re-ranks the merged pool, producing the candidate set forwarded to the reranker.

2.4 Cross-Encoder Reranking

RRF fusion provides a strong initial ranking, but bi-encoder models score query and document independently. A cross-encoder can model their interaction jointly, yielding a more precise relevance estimate at the cost of higher latency. EviGraph-R optionally applies a cross-encoder reranker on the top candidates produced by RRF before returning results.

Given a query q and a candidate chunk d , the cross-encoder computes a scalar relevance score by concatenating them and passing the pair through the model:

$$s(q, d) = \text{CrossEncoder}([q; d]) \in \mathbb{R} \quad (\text{IV.4})$$

The model used is **cross-encoder/ms-marco-MiniLM-L-6-v2** [24], a lightweight MiniLM-based cross-encoder trained on MS MARCO passage relevance judgements. The reranker scores up to 30 RRF candidates in batches of 32, replaces the RRF scores with the cross-encoder scores, and returns the top- k chunks whose score exceeds the minimum threshold $\tau = 0.15$.

2.5 Section Filtering and Budget-Weighted Scoring

After reranking, EviGraph-R applies a *section-aware score boost* to favour chunks whose IMRaD section label matches the sections targeted by the sub-query. The Decomposer assigns each sub-query a list of target sections (e.g. `Methods`, `Results`) based on the nature of the question.

Let $s_{\max}$ be the highest reranker score in the current result list and $\mathcal{T}$ the set of target sections. For each retrieved chunk d , the adjusted score is:

$$s'(d) = \begin{cases} s(d) + \gamma \cdot s_{\max} & \text{if } \text{section}(d) \in \mathcal{T} \\ s(d) & \text{otherwise} \end{cases} \quad (\text{IV.5})$$

where $\gamma = 0.10$ (`section_boost_gamma`). The result list is then re-sorted by s' . This soft nudge does not filter any chunks; it only reorders borderline cases to prefer structurally relevant evidence without discarding candidates from non-targeted sections that may still be pertinent.

3 Agent Roles and Responsibilities

The EviGraph-R pipeline runs through four specialized agents in sequence, where each agent processes the current workflow state and passes its output to the next. This section describes what each agent does, what it takes as input, what it produces, and how it works internally.

3.1 Query Decomposer Agent

The Decomposer Agent is the entry point of the pipeline. Given a user query, it decides whether the question can be answered through a single retrieval target or whether it should be split into several focused sub-queries, each covering a distinct and independently retrievable aspect of the original question.

This decision is handled entirely through a prompted LLM call. The agent is given a structured prompt with few-shot examples illustrating how different query types should be decomposed, which sections they should target, and how budget weights should be distributed.

For each sub-query q_i , the agent produces three elements: a rewritten question, a list of target IMRaD sections from `{Introduction, Methods, Results, Discussion}`, and a budget weight $w_i \in (0, 1]$ with $\sum_i w_i = 1$. Section targets follow a fixed mapping illustrated in the few-shot examples: methodological questions such as “*how does X work*” are directed toward Methods and Introduction, while evaluation-oriented questions target Results and Discussion. Budget weights reflect the relative importance of each sub-query: core sub-queries typically receive weights in $[0.30, 0.40]$, while secondary ones receive weights in $[0.10, 0.20]$. If decomposition is unnecessary, the original query is kept as a single sub-query with $w = 1.0$.

The internal logic of the agent is illustrated in Figure [IV.2](#).

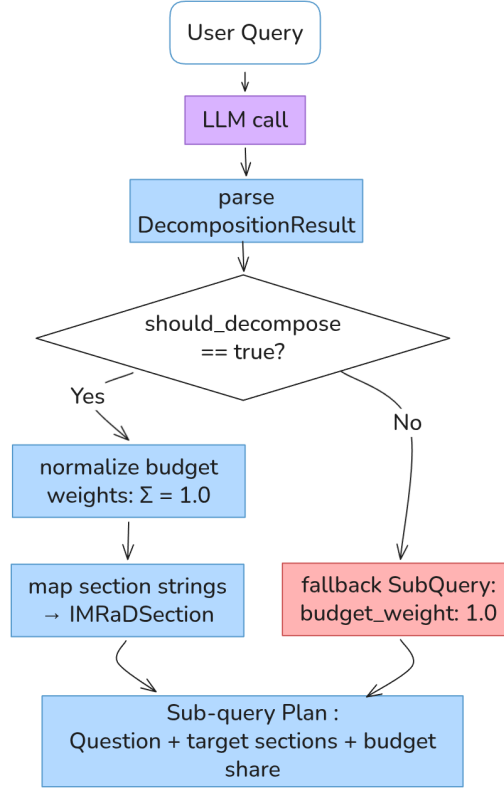


Figure IV.2 – Internal flow of the Decomposer Agent.

3.2 Graph Builder Agent

The Graph Builder Agent is the central component of the EviGraph-R system. It receives the set of retrieved document chunks and constructs the Evidence Property Graph, a structured representation of the scientific knowledge encoded in the retrieved literature. Its operation is divided into four phases, with Phases 1 and 2 executed concurrently to minimise latency.

Phase 1: Citation graph construction. A base graph is built from the citation spans embedded in the retrieved chunks. Each citation span is classified using a fine-tuned version of **SciBERT** [65], trained on the SciCite dataset [18] for scientific citation intent classification. Rather than relying on the original SciCite model, **we fine-tuned our own checkpoint**¹ on the full SciCite training set of 8,243 examples over 8 epochs, achieving a macro F1 of 0.843 and an accuracy of 85.6% on the held-out test set.

The model classifies each citation into one of three intent categories: **background**, **method**, or **result_comparison**. These typed relations form the initial edges of the graph, connecting chunk nodes to the papers they cite.

¹https://huggingface.co/lostelf/scibert_scivocab_uncased_scicite_finetuned

Phase 2: Claim extraction. In parallel with Phase 1, an LLM processes the retrieved chunks and extracts atomic scientific claims. To reduce timeout risks, chunks are grouped into batches of four, and the batch calls are executed in parallel using a thread pool. The extraction prompt follows the principles of **Claimify** [66], a Microsoft Research framework that defines a well-formed claim as atomic, self-contained, and verifiable.

In practice, the prompt asks the model to produce claims containing exactly one assertable fact, replace pronouns with their explicit referents, preserve numerical values and benchmark names, and exclude speculative or future-work statements. Each claim is annotated with a semantic subtype (definition, method, result, or assumption), a relevance label for the sub-query, and an optional `hop_reason` field used in the next phase.

Phase 3: Hop retrieval. Some claims cannot be verified using only the initially retrieved chunks. For example, a chunk may mention a result that is actually supported by a cited paper outside the candidate set. In such cases, the LLM adds a `hop_reason` explaining why the source chunk is insufficient and which cited work should be inspected.

Consider the following contrast:

Claim without hop (self-contained):

“Quantum computing leverages quantum bits (qubits) that exist in superposition to perform parallel computations more efficiently than classical computers.”

[hop_reason: null]

Claim with hop (requires cited paper):

“Quantum error correction using surface codes achieves fault-tolerant computation across 9 distinct error models, including depolarizing and biased noise channels.”

[hop_reason: “The seed chunk cites Fowler 2012 for this result but does not specify which error models were tested. Hop to Fowler 2012 §Methods to confirm.”]

When a `hop_reason` is present, the agent resolves the referenced paper from the inline citation span, identifies its arXiv ID or DOI, and sends a targeted dense retrieval query to Qdrant filtered to that paper. Both the hop limit (50 per graph build) and the concurrency limit (8 simultaneous Qdrant calls) were set empirically: we found that increasing either beyond these values added latency without meaningfully improving the retrieved evidence. The retrieved hop chunks are then added to the graph as additional evidence nodes, extending coverage beyond the initial retrieval results.

IV.3 Agent Roles and Responsibilities

Phase 4: Graph export and rendering. Once all phases are complete, the in-memory graph constructed using **NetworkX** [67], a Python library for the creation and manipulation of complex graph structures, is serialised into a typed `EvidenceGraph` Pydantic object and written back to the shared workflow state. In parallel, the graph is exported to a JSON format compatible with **Cytoscape.js** [68], a JavaScript library for interactive network visualisation, which renders the Evidence Property Graph in the frontend and allows users to explore claim nodes, typed edges, and citation links interactively.

The overall structure of the Graph Builder Agent is depicted in Figure IV.3.

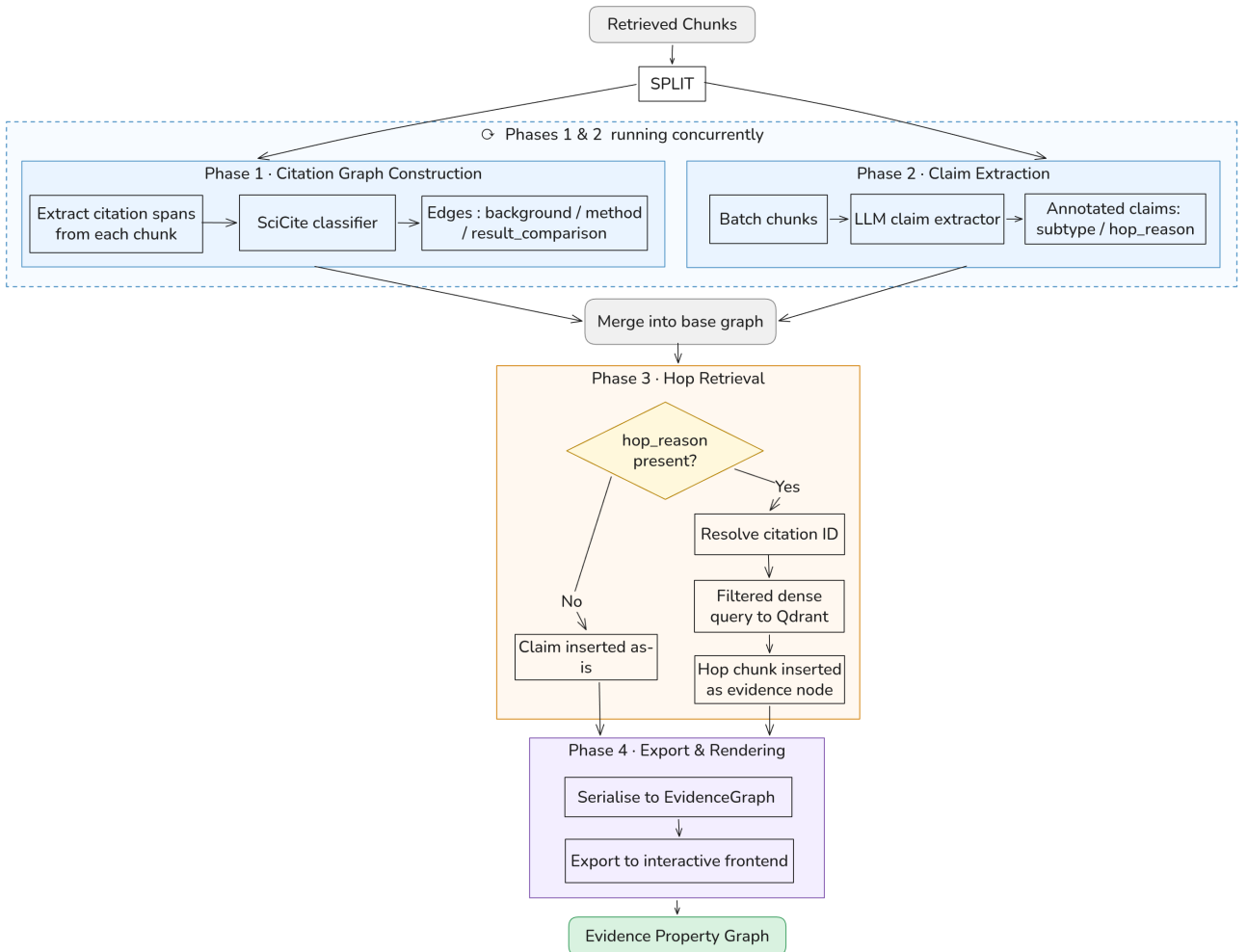


Figure IV.3 – Internal flow of the Graph Builder Agent.

3.3 Judge Agent

The Judge Agent verifies every claim node in the Evidence Property Graph and assigns one of four verdicts: **{Supported, Contradicted, Not_Supported, Inconclusive}**.

Here, **Contradicted** means the evidence explicitly conflicts with the claim, **Not_Supported** means no retrieved evidence could back it up, and **Inconclusive** means evidence exists but leaves the claim unresolved. To handle claims of varying complexity, the agent uses a two-tier verification strategy inspired by AlignScore [69] and VeriTrail [70], selecting the most suitable verifier based on claim complexity and hop depth.

First, each claim is classified as either *atomic-factual*, when it contains numerical values, percentages, named metrics, or versioned model identifiers, or *inferential*. This classification, together with the claim’s hop depth and any contradiction edges in the graph, determines which verification tier is used.

Tier 1: Batched NLI verification. Single-hop claims with no detected contradiction are verified using **sileod/deberta-v3-small-tasksource-nli**, a DeBERTa-v3 model fine-tuned for natural language inference: given a premise and a hypothesis, it predicts whether the premise entails, contradicts, or is neutral toward the hypothesis. Its small size makes it well-suited for lightweight, high-throughput claim verification. Following the idea of AlignScore [69], the linked source chunk is used as the premise and the claim as the hypothesis. A confident entailment prediction gives the verdict **Supported**, while a confident contradiction gives **Contradicted**. If the prediction is `neutral`, the claim is escalated to Tier 2.

Tier 2: LLM judge with backwards graph traversal. Multi-hop claims, claims with contradiction edges, and NLI-neutral cases are verified by the LLM judge. Following the backwards DAG traversal strategy of VeriTrail [70], the LLM receives the claim together with its full evidence trail, collected by walking backwards from the claim node through its source edges to the root chunk nodes. The calls are executed in parallel with a thread pool capped at 8 concurrent threads. The final verdicts, verifier labels, and reasoning traces are written back into the graph as node metadata and typed judge edges, preserving the verification trail.

The routing logic of the Judge Agent is illustrated in Figure IV.4.

3.4 Answer Generator Agent

The Answer Generator Agent synthesises the final response from the verified claims produced by the Judge Agent. Its operation proceeds in three steps.

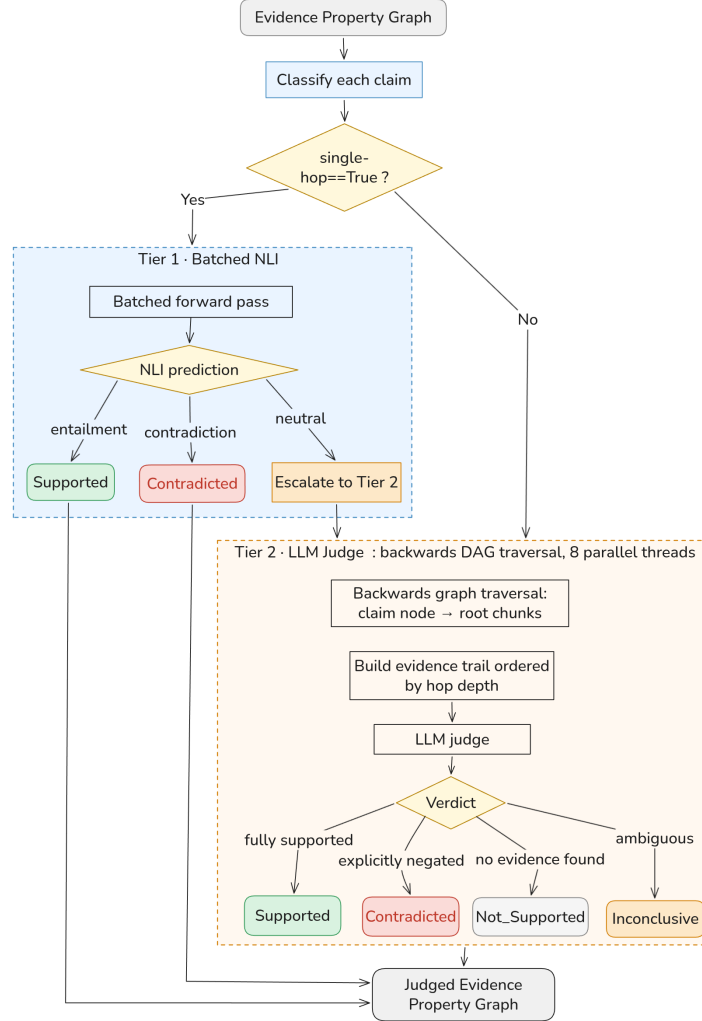


Figure IV.4 – Verification pipeline of the Judge Agent.

Step 1: Claim selection. All claims carrying a verdict of **Supported** or **Inconclusive** are collected and ranked by a composite score:

$$\text{score}(c) = s_{\text{ret}}(c) + 0.3 \cdot \text{sim}(c) - 0.5 \cdot \delta(c) \quad (\text{IV.6})$$

where $s_{\text{ret}}(c)$ is the retrieval score of the source chunk, $\text{sim}(c)$ is the maximum Jaccard similarity between the claim and any sub-query text, and $\delta(c) = 1$ if the claim verdict is **Inconclusive** and 0 otherwise.

Step 2: Answer generation. The ranked claims are submitted to the LLM, which produces a fluent, coherent response in which each sentence is explicitly linked to a list of claim

indices (`claim_refs`) from which it was derived.

Step 3: Citation assembly. Each generated sentence is linked to `Citation` objects derived from its referenced claims. Each citation stores the source paper identifier, chunk identifier, section title, SciCite relation label, and original claim text. If a sentence relies on claims connected to a contradiction edge in the Evidence Property Graph, a conflict flag is added. The final output is a `FinalAnswer` object containing the answer text and complete per-sentence citation traces back to the source literature.

The complete flow of the Answer Generator Agent is shown in Figure IV.5.

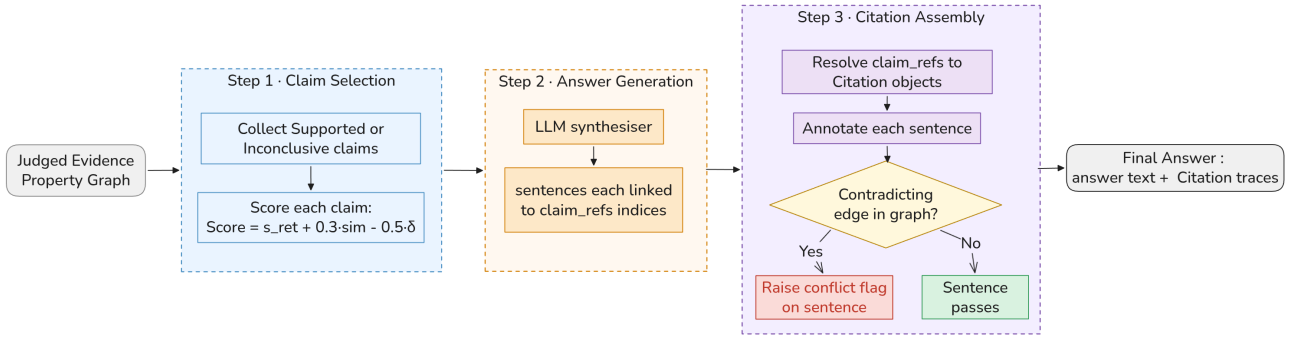


Figure IV.5 – Flow of the Answer Generator Agent.

4 End-to-End Walkthrough

This section traces a single query through every stage of the EviGraph-R pipeline, from decomposition to final answer, using a concrete running example. Each step shows the input received, the transformation applied, and the output passed to the next stage.

4.1 Query Decomposition

Consider the following user query:

“What is contrastive learning and how does it perform compared to supervised pre-training on biomedical text classification?”

This query contains two distinct retrievable intents: a definitional question about contrastive learning, and an empirical comparison question. The Decomposer Agent detects both intents and produces the following decomposition:

#	Sub-query	Target sections	Weight
1	What is contrastive learning?	Introduction, Methods	0.35
2	How does contrastive learning compare to supervised pre-training on biomedical text classification?	Results, Discussion	0.65

Tableau IV.1 – Decomposer output for the running example query.

Sub-query 2 receives the higher budget weight (0.65) because answering a comparative empirical question requires more evidence than a definitional one. Both the weights and section targets are determined by the LLM through the decomposition prompt, which instructs it to allocate more budget to empirically heavier sub-queries and direct each one toward the sections where the relevant evidence is most likely to appear.

4.2 Sub-query Retrieval

Each sub-query is embedded with BGE-M3, searched against the Qdrant index using hybrid dense+sparse retrieval with RRF fusion, and then reranked by the cross-encoder. The top- k chunks allocated to each sub-query are proportional to its budget weight. The table IV.2 shows one representative chunk retrieved per sub-query (SQ).

SQ	Paper	Chunk excerpt	Section
1	SimCLR (Chen et al., 2020)	“Contrastive learning is a self-supervised objective that trains an encoder to bring representations of augmented views of the same instance closer together while pushing apart representations of different instances.”	Methods
2	BioBERT-CL (Lee et al., 2022)	“On the HoC benchmark, our contrastive pre-training approach achieves 87.3% F1, outperforming standard BERT fine-tuning (84.1%) and BioBERT (85.6%) on the same task.”	Results

Tableau IV.2 – Representative retrieved chunks per sub-query.

In practice, the system retrieves several chunks per sub-query. However, Table IV.2 shows only one representative chunk per sub-query for readability.

4.3 Evidence Graph Construction

The Graph Builder Agent processes both chunks in parallel. Phase 1 builds the citation graph. Phase 2 extracts claims. The resulting graph contains two types of nodes: **chunk nodes** (the raw retrieved passages) and **claim nodes** (atomic facts extracted from them). Edges carry a typed relation label.

Extracted claims. From Chunk 1 (SimCLR, Methods), the LLM extracts a self-contained definitional claim that requires no further evidence:

Claim A *“Contrastive learning trains an encoder to bring representations of augmented views of the same instance closer together while pushing apart representations of different instances.”*

subtype: definition hop_reason: none

From Chunk 2 (BioBERT-CL, Results), the LLM extracts a result claim that references a benchmark whose composition is defined in a separate cited paper:

Claim B *“Contrastive pre-training achieves 87.3% F1 on the HoC benchmark, outperforming standard BERT fine-tuning (84.1%) and BioBERT (85.6%).”*

subtype: result hop_reason: missing_scope_context

look_for: ``HoC benchmark definition and task description``

Claim B triggers hop retrieval: the agent resolves the inline citation for the Hallmarks of Cancer (HoC) benchmark paper, identifies its arXiv ID, and issues a filtered dense retrieval query to Qdrant scoped to that paper. The top matching chunk is added to the graph as an additional evidence node (Chunk 3 in this simplified illustration; in practice the full graph contains many more chunk and claim nodes across all retrieved papers).

Graph structure. Figure IV.6 shows the resulting graph for this example. It is a minimal mock intended only to illustrate the three node types, the edge relation labels, and the hop structure. A real evidence graph over a full query contains tens of chunk nodes, multiple claim nodes per chunk, and several hop edges.

4.4 Claim Verification

The Judge Agent processes each claim node and assigns a verdict.

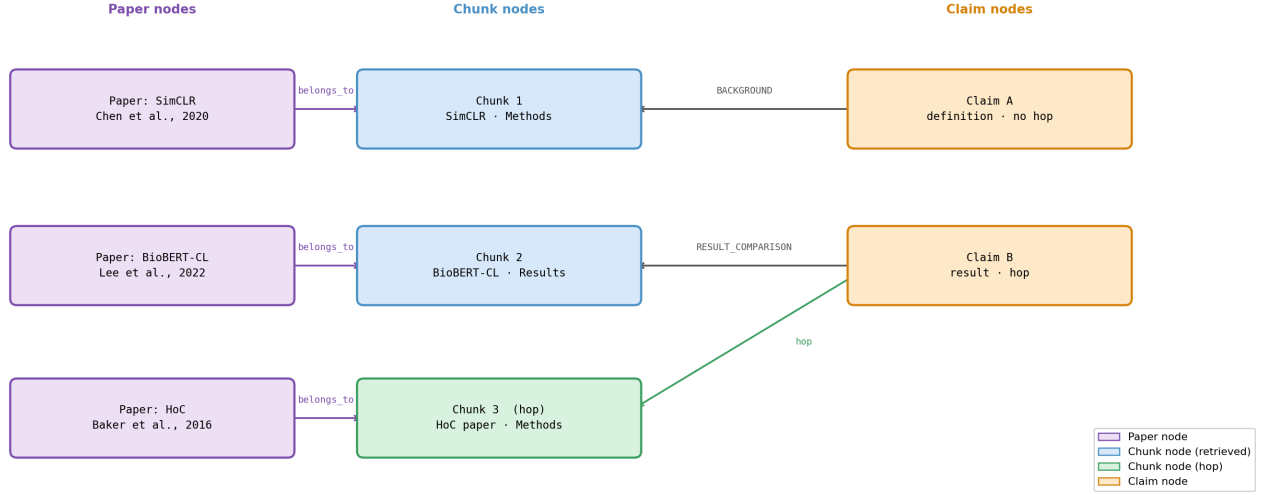


Figure IV.6 – Minimal evidence graph for the running example.

Claim A : NLI succeeds (Tier 1). Claim A is a single-hop inferential claim with no contradiction detected. It is routed to the batched NLI verifier. The verifier receives:

- **Hypothesis:** *“Contrastive learning trains an encoder to bring representations of augmented views of the same instance closer together while pushing apart representations of different instances.”*
- **Premise:** the full text of Chunk 1.

The NLI model returns entailment with high confidence. Verdict: **Supported**.

Claim B : escalated to LLM judge (Tier 2). Claim B contains specific numerical values (87.3%, 84.1%, 85.6%) and references a hop chunk, making it a multi-hop atomic-factual claim. It is routed directly to the LLM judge. The LLM receives the claim text and the backwards evidence trail:

- **Iteration 1** (Chunk 2, RESULT_COMPARISON): *“On the HoC benchmark, our contrastive pre-training approach achieves 87.3% F1, outperforming standard BERT fine-tuning (84.1%) and BioBERT (85.6%).”*
- **Iteration 2** (Chunk 3, hop): *“The Hallmarks of Cancer (HoC) corpus consists of 1,499 PubMed abstracts annotated for ten cancer hallmark categories, used as a multi-label classification benchmark.”*

The LLM finds that the numerical result is confirmed in Iteration 1 and that the benchmark context is confirmed in Iteration 2. Verdict: **Supported**.

4.5 Answer Generation and Citations

Both supported claims are collected, scored, and submitted to the Answer Generator. The LLM produces the following two-sentence answer, with each sentence explicitly linked to the claims it draws from:

Generated answer

Contrastive learning is a self-supervised training objective that optimises an encoder to bring representations of different augmented views of the same instance closer in embedding space while separating representations of distinct instances [1]. ([Claim A → SimCLR, Methods, chunk a3f2c1])

When applied to biomedical text, contrastive pre-training outperforms both standard BERT fine-tuning and BioBERT on the Hallmarks of Cancer (HoC) classification benchmark, achieving 87.3% F1 against 84.1% and 85.6% respectively [2]. ([Claim B → BioBERT-CL, Results, chunk b7d9e4; HoC paper, Methods, chunk c1a8f2])

The system returns a `FinalAnswer` object in which each sentence carries a `Citation` record specifying the source paper identifier, the chunk identifier, the IMRaD section, the Sci-Cite relation label, and the original claim text, providing a complete and auditable evidence trail from answer back to the source literature.

Please note that this example is intentionally simplified: in a real EviGraph-R response, citations reference actual chunk identifiers and paper arXiv IDs rather than symbolic claim labels, each sentence typically draws on multiple chunks from different papers, and the answer spans a larger pool of verified claims.

Conclusion

This chapter detailed the design of EviGraph-R as a modular multi-agent architecture, from hybrid retrieval and evidence graph construction to claim verification and citation-grounded answer generation. The walkthrough showed how each component transforms the shared workflow state and contributes to producing transparent, verifiable answers grounded in scientific literature.

Chapter V

System Evaluation and Integration

Summary

1	Index and Retrieval Evaluation	51
1.1	Experimental Setup	51
1.2	Retrieval Metrics	52
1.3	Retrieval Performance and Interpretation	53
2	System Evaluation	55
2.1	Evaluation Framework	55
2.2	Evaluation Metrics	57
2.3	Ablation Study	57
2.4	Baselines Evaluation	59
2.5	Results and Interpretation of Findings	59
3	Software Integration	60
3.1	Backend: FastAPI Service	60
3.2	System Monitoring: Langfuse Tracing	62
3.3	Frontend: ReactJS Interface	62
3.4	Pip Package	67

Introduction

This chapter evaluates EviGraph-R across two complementary dimensions. We begin with an intrinsic assessment of the retrieval subsystem, benchmarking our hybrid BGE-M3 index against the SQuAI baseline. We then move to a full end-to-end system evaluation through ablation and baseline comparisons. The chapter closes with an overview of the software integration, covering the FastAPI backend, the ReactJS interface and the pip package.

1 Index and Retrieval Evaluation

This section benchmarks EviGraph-R’s retrieval subsystem against SQuAI, covering the experimental setup, the metrics used and the results.

1.1 Experimental Setup

Compared systems: We compare two indexing systems:

SQuAI [12] indexes paper abstracts using `intfloat/e5-large-v2` dense embeddings in a FAISS Flat index, combined with BM25 ($k_1=1.5$, $b=0.75$, $\delta=0.5$), fused via linear interpolation:

$$\text{score} = \alpha \cdot s_{\text{dense}} + (1 - \alpha) \cdot s_{\text{BM25}}, \quad \alpha = 0.5 \quad (\text{V.1})$$

where s_{dense} and s_{BM25} are the normalised retrieval scores and α controls their relative weight.

EviGraph-R indexes full paper subsections using BAAI/bge-m3, which produces dense and sparse representations in a single forward pass, stored in a Qdrant collection. Retrieval lists are merged via Reciprocal Rank Fusion (RRF):

$$\text{score}_{\text{RRF}} = \sum_i \frac{1}{k + \text{rank}_i}, \quad k = 60, \text{ prefetch top-100} \quad (\text{V.2})$$

where rank_i is the candidate’s rank in the i -th modality list and k is a smoothing constant. Both systems index the same unarXive corpus of approximately 2.28 million arXiv papers.

Evaluation corpus and clustered sampling: A naive random sample over more than 2M papers would be misleading, since both systems could retrieve papers that are genuinely more relevant than the designated gold paper if the question is too generic. To avoid this, we use a **clustered sampling** strategy. Papers are selected from topically coherent clusters, where each paper acts as a plausible hard negative for the others. Questions are also grounded in specific body sections rather than abstracts, making them answerable only from the gold paper.

We define three broad scientific domains: CS, Physics, and Mathematics, each divided into three sub-clusters. This gives nine clusters in total, as shown in Table V.1. Five papers are sampled per cluster, resulting in **45 evaluation papers**.

V.1 Index and Retrieval Evaluation

Tableau V.1 – Evaluation corpus: nine topical clusters across three scientific domains.

Domain	Cluster A	Cluster B	Cluster C
CS	LLM & prompt engineering	Databases & retrieval	AI for healthcare
Physics	Quantum computing	Astrophysics / cosmology	Condensed matter
Math	Number theory	Differential geometry	Combinatorics

Question design: Each of the 45 papers produces three questions, giving 135 evaluation queries. After quality filtering, where questions that were too ambiguous, malformed, or unanswerable from the retrieved chunks were discarded, **113 queries** were retained for evaluation. Questions are generated by Llama-3.3-70B from body text retrieved from Qdrant (`chunk_type=subsection`), not from abstracts, and follow the 3 types shown in Table V.2.

Tableau V.2 – Question types used in the evaluation.

Type	Source	Purpose
section	Best body section (methods / results)	Primary signal: requires full-text
fullpaper	Same section, different angle	Tests contribution-level retrieval
abstract	Abstract text	Control: SQuAI’s native mode

The `section` and `fullpaper` questions are intentionally difficult for SQuAI because it indexes only abstracts, while EviGraph-R indexes the full subsection text from which these questions are derived.

1.2 Retrieval Metrics

All primary metrics are computed at $k \in \{1, 5, 10\}$. Let r denote the rank of the gold paper in the retrieved list (undefined if not found within k).

Tableau V.3 – Primary retrieval metrics.

Metric	Definition
Hit@ k	1 if gold <code>paper_id</code> appears in the top- k results, else 0
MRR@ k	$\mathbb{E}\left[\frac{1}{r}\right]$ if $r \leq k$, else 0
NDCG@ k	Normalised DCG; relevance scores: 1.0 (gold), 0.5 (partial), 0.0 (miss)
MAP@ k	Mean Average Precision across rank positions up to k

In addition to the primary metrics, we report the supplementary measures listed in Table V.4. Section Hit@ k is exclusive to EviGraph-R since SQuAI operates at the abstract level

and carries no section-granularity metadata.

Tableau V.4 – Supplementary retrieval metrics.

Metric	Definition	System
Precision@ k	$ \text{gold papers in top-}k /k$	Both
Avg. rank of gold	Mean rank position when the gold paper is found	Both
Section Hit@ k	Retrieved chunk originates from the correct section	EviGraph-R
Latency p50 (ms)	Median query wall-clock time	Both
Latency p95 (ms)	95th-percentile query wall-clock time	Both

All metrics are broken down along two dimensions: scientific domain (CS / Physics / Math) and question source (section / fullpaper / abstract).

1.3 Retrieval Performance and Interpretation

We present here the retrieval results obtained on the 113-query evaluation set and compares the performance of EviGraph-R against the SQuAI baseline.

Figure V.1 compares Hit@1, Hit@5, and MRR@10 across four evaluation splits on the shared set of 113 evaluation queries.

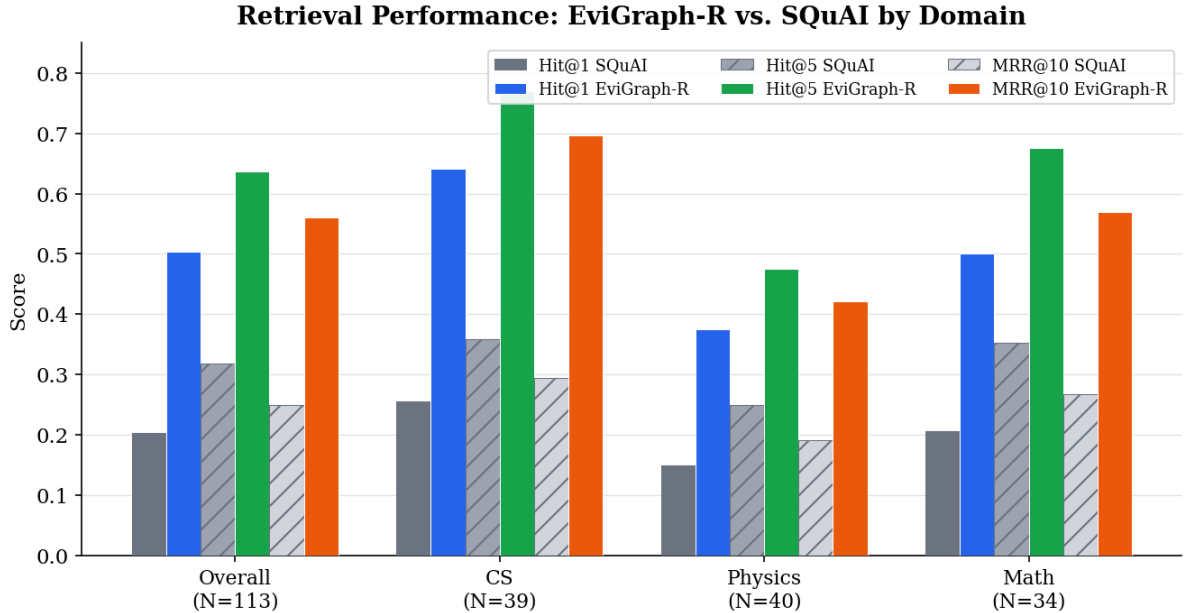


Figure V.1 – Retrieval performance of EviGraph-R vs. SQuAI across three domains.

Table V.5 reports the full set of primary metrics at $k \in \{1, 5, 10\}$ for both systems.

V.1 Index and Retrieval Evaluation

Tableau V.5 – Primary retrieval metrics at $k = 1, 5, 10$ for EviGraph-R vs. SQuAI (overall).

Metric	SQuAI (baseline)			EviGraph-R (ours)		
	$k=1$	$k=5$	$k=10$	$k=1$	$k=5$	$k=10$
Hit@ k	0.204	0.319	0.345	0.504	0.637	0.646
MRR@ k	0.204	0.247	0.250	0.504	0.560	0.561
NDCG@ k	0.204	0.138	0.142	0.504	0.303	0.305
MAP@ k	0.204	0.247	0.250	0.504	0.560	0.561

Overall, EviGraph-R outperforms SQuAI across all primary metrics. Hit@1 increases from 0.20 to 0.50, while Hit@5 rises from 0.32 to 0.64, showing a clear improvement at the scale of 2.28M papers. This gain mainly comes from three factors: full-subsection indexing gives access to evidence that abstract-only retrieval misses, BGE-M3 combines dense and sparse signals in one model, and RRF provides a stable way to fuse rankings without relying on raw score calibration.

Table V.6 breaks down Hit@ k and MRR@10 by domain and question type, showing where each system performs best.

Tableau V.6 – Hit@ k and MRR@10 broken down by scientific domain and question type.

Split	Question type	SQuAI			EviGraph-R		
		Hit@1	Hit@5	MRR@10	Hit@1	Hit@5	MRR@10
CS	section	0.077	0.077	0.085	0.615	0.692	0.654
	fullpaper	0.231	0.308	0.256	0.538	0.769	0.654
	abstract	0.462	0.692	0.541	0.769	0.846	0.785
Physics	section	0.000	0.071	0.036	0.214	0.357	0.274
	fullpaper	0.071	0.071	0.071	0.357	0.429	0.393
	abstract	0.417	0.667	0.512	0.583	0.667	0.625
Math	section	0.091	0.091	0.091	0.455	0.545	0.485
	fullpaper	0.167	0.167	0.167	0.417	0.500	0.458
	abstract	0.364	0.818	0.556	0.636	1.000	0.776

The improvement is consistent across all domains and question types. The strongest gains appear in CS, where Hit@1 on `section` questions rises from 0.077 to 0.615. Physics shows smaller gains, likely due to less consistent section naming in some papers. EviGraph-R also performs better on `abstract` questions, even though this is SQuAI’s native setting, suggesting that the hybrid BGE-M3 setup captures abstract-level signals effectively. The Math results,

especially the Hit@5 of 1.000 on abstract questions, further show the strength of hybrid retrieval in terminology-rich domains.

2 System Evaluation

This section evaluates EviGraph-R end-to-end by presenting the evaluation framework, dataset construction, metrics, ablation study, baseline comparison, and final interpretation of the results.

2.1 Evaluation Framework

Framework and Tooling: We use **DeepEval** [71] as our evaluation framework, following the same approach established in SQuAI. DeepEval provides production-ready implementations of standard RAG metrics, a Synthesizer for automated question generation from any text corpus, and G-Eval for defining custom LLM-as-judge scoring criteria.

Dataset Construction: All questions are sampled from the unarXive arXiv collection indexed in Qdrant: approximately 2.28M papers represented as 69M chunks. No external documents are introduced.

Questions are stratified across four discipline groups derived from arXiv category prefixes. The realised distribution across the 268 evaluated questions is shown in Table V.7.

Tableau V.7 – Domain distribution of the benchmark dataset.

Domain	n	Share
Computer Science	68	25.4%
Physics	78	29.1%
Mathematics	66	24.6%
Statistics / Other	56	20.9%
Total	268	100%

Question categories: The dataset is divided into four categories, each testing a different capability of the pipeline.

Category 1: Single-paper synthesis. Three subsection chunks from the same paper are grouped into one context, and the synthesiser generates a broad thematic question. This tests

basic retrieval and answer generation, and serves as a regression baseline shared with SQuAI and standard RAG.

Category 2: Cross-section synthesis. Two chunks are sampled from different IMRaD sections of the same paper, such as Methods and Results. This tests whether the Decomposer can route sub-queries to the right sections. Chunks with noisy or missing section labels are excluded.

Category 3: Citation expansion. A citing chunk `chunk_A` is paired with a cited chunk `chunk_B` resolved through a `cite_span`. Questions are generated with a bridge-constrained prompt adapted from HotpotQA [72] and MedHop [73], requiring both chunks to answer. A single-hop answerability filter [74] removes questions answerable from only one chunk. Contexts are oversampled $2\times$ to account for the expected $\approx 40\%$ discard rate. This category directly tests citation expansion.

Category 4: Thematic multi-paper synthesis. Three chunks are selected from three different papers in the same arXiv category group, with cosine similarity between 0.50 and 0.80. Since no citation link is required, multi-source necessity is enforced using the same single-hop answerability filter. This is the most complete test of the full pipeline.

Question style diversity. To avoid repetitive question templates, we use the NFQ taxonomy of Bolotova et al. [75], covering REASON, EVIDENCE-BASED, COMPARISON, INSTRUCTION, EXPERIENCE, and DEBATE questions. We also add FACTOID for short single-value questions. Each category targets 10 questions per style, with the final distribution shown in Table V.8.

Tableau V.8 – NFQ style distribution across the benchmark dataset.

NFQ Style	Cat. 1	Cat. 2	Cat. 3	Cat. 4	Total
REASON	6	8	10	10	34
EVIDENCE-BASED	10	10	10	10	40
COMPARISON	10	8	10	10	38
INSTRUCTION	10	10	10	10	40
EXPERIENCE	10	10	10	10	40
FACTOID	8	8	10	10	36
DEBATE	10	10	10	10	40
Total	64	64	70	70	268

Immunity to corpus incompleteness. The main metrics, Answer Relevance, Contextual Relevance, and Faithfulness, evaluate what the system actually returns rather than compar-

ing retrieved chunks to a fixed gold set. This avoids penalising the system when it retrieves different but still valid evidence from a large corpus with many relevant chunks. Only Claim Coverage uses a gold expected output, generated directly from the same corpus during question construction.

2.2 Evaluation Metrics

We evaluate every system configuration using five complementary metrics. The first three: Answer Relevance, Contextual Relevance, and Faithfulness, reproduce the metrics used in SQuAI [12], enabling direct numeric comparison across generations. The remaining two are custom metrics designed specifically for EviGraph-R’s structural output.

Answer Relevance. Whether the answer directly addresses the question, regardless of the evidence used. A low score signals retrieval failure or answer drift.

Contextual Relevance. Whether the retrieved chunks are topically relevant to the question, evaluating retrieval quality independently of the final answer.

Faithfulness. Whether the claims in the answer are grounded in the retrieved context. A system can score high on Answer Relevance while fabricating content; Faithfulness catches this.

Claim Coverage. A custom G-Eval metric checking whether the answer covers the key factual claims in the gold expected answer.

Attribution Faithfulness. A custom G-Eval metric verifying that each inline citation actually supports the sentence it is attached to a failure mode standard Faithfulness misses.

Aggregate score. We report the mean of all five metrics as $mean_5$ per configuration and question category.

2.3 Ablation Study

The ablation study disables one component at a time and measures the resulting delta against the full EviGraph-R pipeline. Variants are organised by agent group.

Agent 1: Query Decomposer.

- **A1.1 No decomposition.** The raw query is sent directly to the retriever, without generating sub-queries. This removes section-aware routing and forces the system to retrieve from a single query embedding instead of aspect-specific queries targeted at IMRaD sections. The expected impact is a drop on Category 2, where section routing is essential.
- **A1.2 No budget weights.** All sub-queries receive the same retrieval budget (weight = $1/n$). This removes the ability to prioritise more evidence-demanding aspects and is expected to reduce Claim Coverage on compound questions.

Node: Hybrid Retriever.

- **R1 Dense only.** The BGE sparse retrieval path is removed. Since sparse retrieval captures exact scientific terms that dense embeddings may smooth over, this variant is expected to reduce Contextual Relevance on keyword-heavy queries.
- **R2 Sparse only.** The dense embedding path is removed. Since dense retrieval captures semantic similarity, this variant is expected to reduce recall for paraphrased or conceptually related evidence.
- **R3 No section-aware boosting.** The IMRaD section boost is disabled ($\gamma = 0$). Without this boost, noisy or less relevant sections are less effectively de-ranked, which should mainly affect Contextual Relevance and Faithfulness on Category 2.

Agent 2: Evidence Graph Builder.

- **G1 Flat chunks.** The evidence graph is replaced by a flat ordered list of retrieved chunks, with no claim extraction or citation edges. Removing this relational structure is expected to reduce Faithfulness and Claim Coverage on Categories 3 and 4.
- **G2 No citation expansion.** Citation expansion is disabled (`hop_enabled = false`), so only the initially retrieved chunks are used. This directly tests EviGraph-R’s main contribution and is expected to cause the largest drop on Category 3, where questions require evidence from both the citing and cited paper.

Agent 3: Reasoning Judge.

- **J1 No judge.** Extracted claims are passed directly to the Answer Generator without verification. This may allow unsupported claims and misattributed citations to appear in the final answer.

- **J2 NLI only.** The LLM judge is disabled, and NLI is used for all claims, including complex and cross-paper ones. Since NLI is weaker for multi-document reasoning, this is expected to reduce Attribution Faithfulness on Category 3.
- **J3 LLM only.** NLI routing is disabled, and the LLM judge verifies all claims. Citation accuracy should remain stable, but latency is expected to increase.

2.4 Baselines Evaluation

We compare EviGraph-R against two baselines with different levels of pipeline complexity.

Standard RAG. This baseline uses a simple flat retrieval pipeline: the raw query is embedded with BGE-M3, the top- k chunks are retrieved from Qdrant using dense similarity, and the retrieved text is passed directly to an LLM. It includes no decomposition, reranking, evidence graph, or claim verification, and therefore represents the simplest practical baseline.

SQuAI. SQuAI is our predecessor system. It uses hybrid retrieval and structured answer generation, but does not build an evidence graph, perform citation expansion, or apply an adaptive claim verification judge. Its results on unarXive provide a directly comparable reference point.

2.5 Results and Interpretation of Findings

Table V.9 reports mean₅ per category for each ablation variant, alongside the two external baselines and the full EviGraph-R system.

Overall, the results show that EviGraph-R clearly improves over Standard RAG and SQuAI, confirming the benefit of adding structured retrieval, evidence graph construction, citation expansion, and claim verification. The strongest drops occur for **R2 Sparse only** and **G1 Flat chunks**, showing that dense semantic retrieval and the evidence graph are essential for scientific QA.

The retrieval ablations confirm that hybrid retrieval is more robust than relying on only one retrieval path: dense-only retrieval remains competitive, but performs below the full system, while sparse-only retrieval drops sharply. The judge variants show a clear accuracy–latency trade-off, with **J3 LLM only** improving some scores but strongly increasing latency.

Interestingly, some variants slightly outperform the full system, especially **A1.1 No decomposition** and **G2 No citation expansion**. This suggests that decomposition and citation expansion can sometimes introduce noise, and should be further refined. Overall, the ablation

Tableau V.9 – Ablation and baselines results.

Configuration	Cat. 1	Cat. 2	Cat. 3	Cat. 4	Overall Avg.	Latency (s)
Standard RAG	0.6431	0.6847	0.7084	0.7168	0.6889	24.1
SQuAI	0.7452	0.7549	0.7394	0.7698	0.7524	67.3
EviGraph-R (full)	0.7724	0.7794	0.8070	0.8294	0.7983	49.8
A1.1 No decomposition	0.7763	0.8179	0.8075	0.8271	0.8076	44.8
A1.2 No budget weights	0.7651	0.7909	0.8174	0.8148	0.7981	49.0
R1 Dense only	0.7507	0.7801	0.7811	0.7917	0.7765	37.0
R2 Sparse only	0.3176	0.3073	0.3466	0.3390	0.3283	4.5
R3 No section boosting	0.7685	0.7852	0.8163	0.8033	0.7941	55.1
G1 Flat chunks	0.4796	0.4635	0.5070	0.4864	0.4845	31.0
G2 No citation expan.	0.7673	0.8007	0.8174	0.8295	0.8047	44.5
J1 No judge	0.7778	0.7729	0.8134	0.8108	0.7947	47.5
J2 NLI only	0.7804	0.7876	0.8124	0.8160	0.7998	48.7
J3 LLM only	0.7703	0.7844	0.8115	0.8297	0.8001	77.2

study confirms the importance of the main EviGraph-R components while identifying retrieval control and citation expansion as future improvement points.

3 Software Integration

This section describes the software stack built around EviGraph-R, covering the FastAPI backend, the ReactJS interface and the pip package.

3.1 Backend: FastAPI Service

The backend is a **FastAPI** service served by Uvicorn and deployed in Docker. A single `docker compose up` command starts the API with Qdrant on a shared network. At startup, the service loads the vector store, **BAAI/BGE-M3** embeddings, and the **DeBERTa-v3** NLI model to avoid first-request cold starts.

All pipeline operations are exposed under the `/api/v1` prefix:

- `GET /health`: checks Qdrant connectivity and reports the active collection and embedding model.
- `POST /api/v1/query`: runs the full pipeline synchronously and returns the answer, citations, evidence graph, scorecard, and timings.

- GET `/api/v1/query/stream`: streams the main pipeline stages as SSE events, enabling the UI to update progressively.
- POST `/api/v1/graph/render`: renders a serialised evidence graph as an embeddable Cytoscape.js HTML page.

Figure V.2 shows the automatically generated FastAPI documentation at `/docs`, while Figure V.3 shows a sample healthy `/health` response.

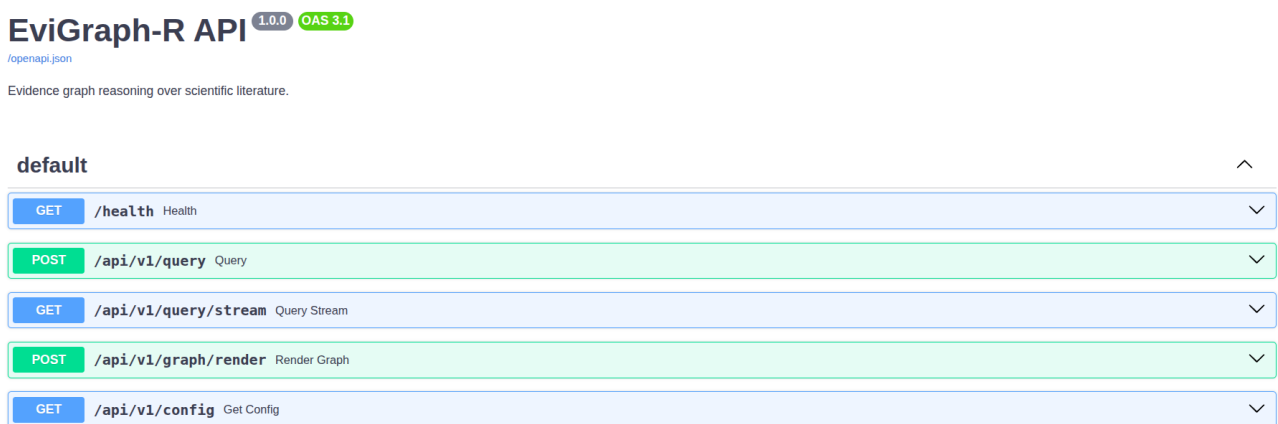


Figure V.2 – The EviGraph-R interactive API documentation at `/docs`.

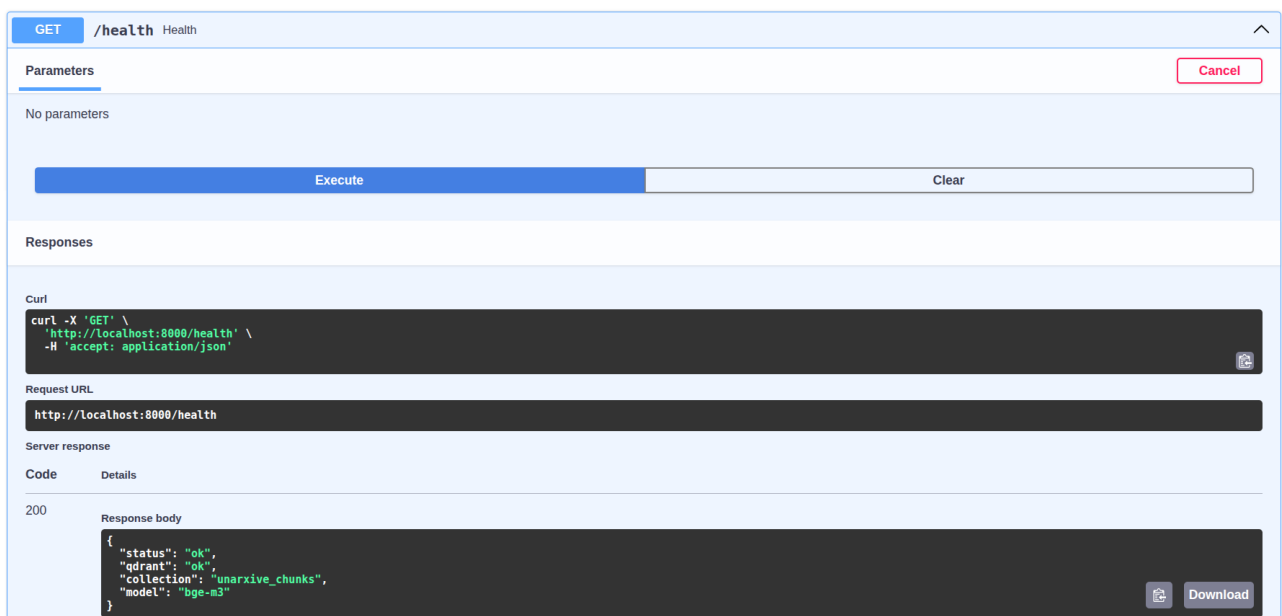


Figure V.3 – GET `/health` response confirming Qdrant reachability and the loaded embedding model.

3.2 System Monitoring: Langfuse Tracing

EviGraph-R uses **Langfuse** to trace each query as a LangGraph run, recording agent-level spans with their latency and inputs/outputs for debugging.

Figure V.4 shows that retrieval is the main bottleneck at 14.59s, followed by graph construction and answer generation. Decomposition and judgment are comparatively lightweight, indicating that most latency comes from retrieval and synthesis.

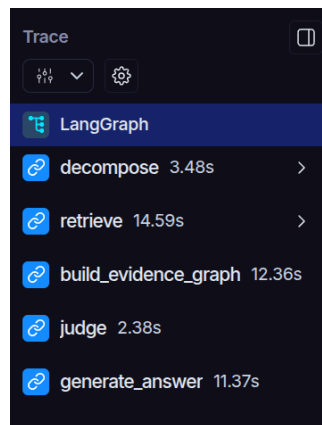


Figure V.4 – Langfuse trace showing per-agent latency for a sample EviGraph-R query.

3.3 Frontend: ReactJS Interface

The frontend is a single-page application built with **React 18**, **TypeScript**, **Vite**, and **Tailwind CSS**. In production, an **Nginx** container serves the compiled interface and proxies `/api` requests to the backend. The UI uses **SSE streaming**, allowing pipeline stages to appear progressively as they complete.

The interface is organised around a single *Playground* view with three main areas, as shown in Figures V.5–V.10. The **sidebar** stores query history and allows previous runs to be restored. The **input bar** contains the research question field and retrieval settings. The **result area** displays live pipeline progress, intermediate outputs, the final answer with citation drawers, a scorecard, and an interactive Cytoscape.js evidence graph.

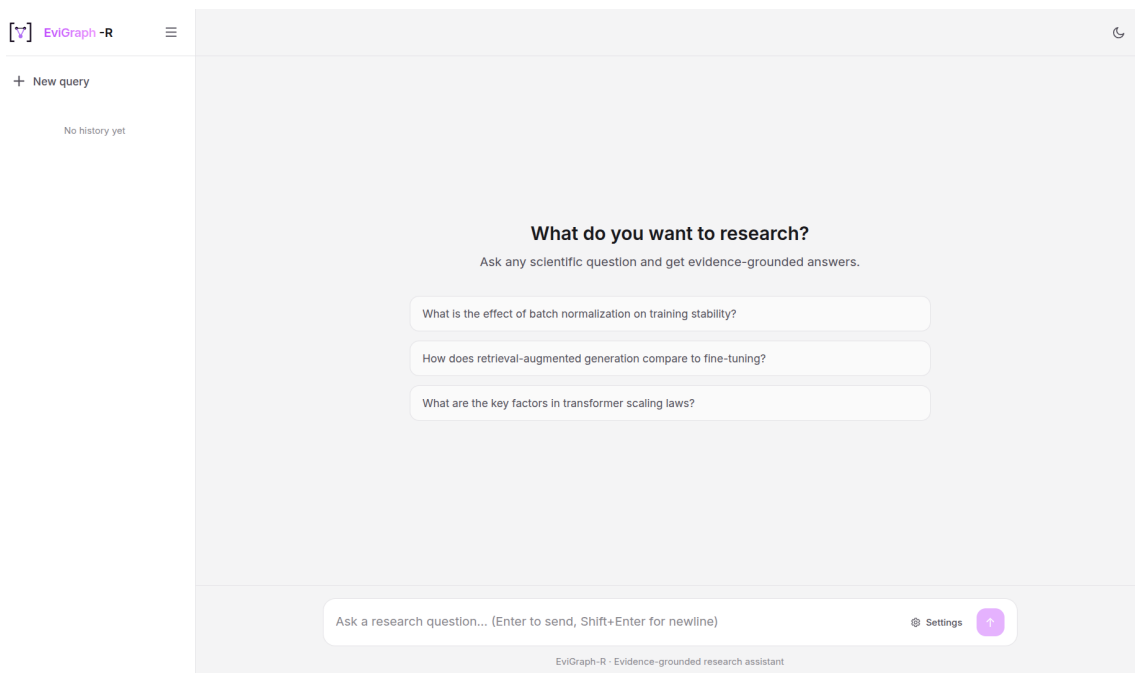


Figure V.5 – Landing screen with query cards, collapsible sidebar, and the input bar.

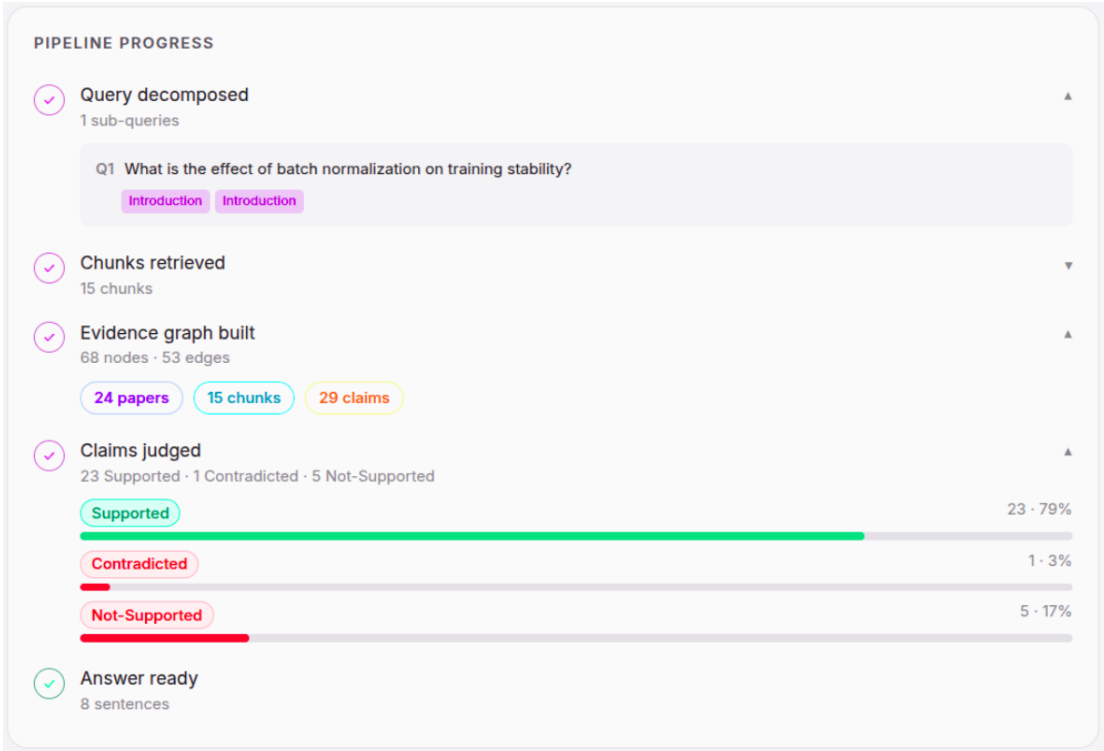


Figure V.6 – Pipeline progress tracker during a live query.

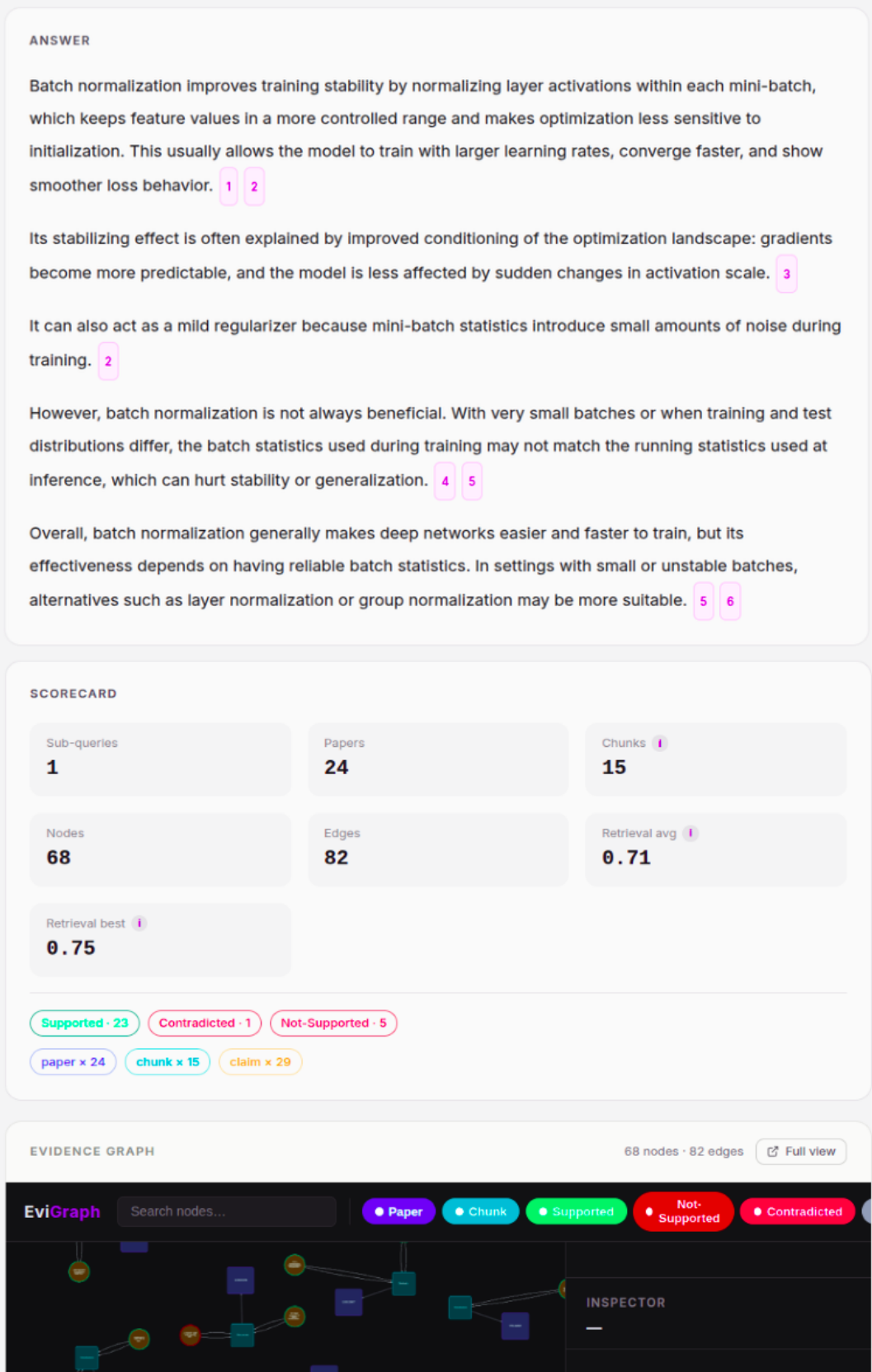


Figure V.7 – Result panel after query completion.

[1]

Citation

×

ARXIV ID

2102.10929

PAPER TITLE

Deep Learning for Robust Motion Segmentation with Non-Static Cameras

SECTION

Results

CLAIM

Batch normalization has a regularization effect.

CITATION INTENT

BACKGROUND

CHUNK CONTENT

Batch normalization as introduced in Section REF aims at accelerating training. Besides this, batch normalization has a regularization effect . The impact on the training lossThe loss represents an inaccuracy calculated by the objective function, and thus is unit-less. of using batch normalization is shown in Figure REF . Applying batch normalization leads to a smoother training loss, which is on average smaller than the training loss without batch normalization. [Figure: Effect of batch normalization on the training loss.]This leads to a faster convergence of the validation loss and therefore to an earlier stopping of the training process due to the Early Stopping technique (see Section REF).

Figure V.8 – Citation side drawer displaying chunk text, paper metadata and IMRaD.

V.3 Software Integration

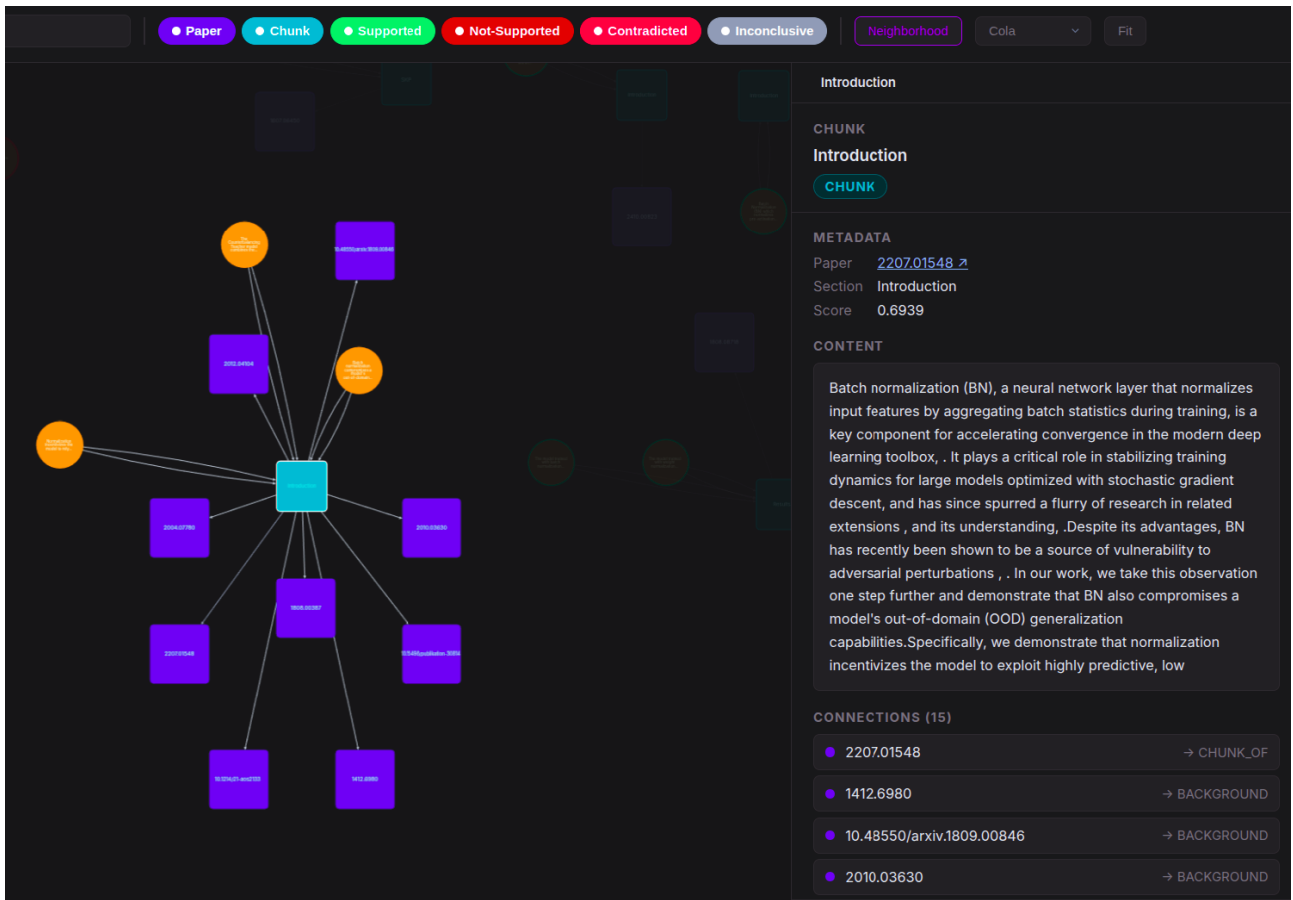


Figure V.9 – Interactive evidence graph rendered with Cytoscape.js.

The settings panel is titled 'PIPELINE SETTINGS' and includes a 'Reset to defaults' link. It features two sliders: 'Top K' (Chunks retrieved per sub-query) set to 15, and 'Score threshold' (Min RRF score to keep a chunk) set to 0.8. A toggle for 'Claim-guided citation expansion' is currently 'Enabled'. Below, 'Target IMRaD sections' are listed as 'Introduction', 'Methods', 'Results', and 'Discussion', with a note to 'Leave all unselected to search all sections.' At the bottom, there is a text input for 'Ask a research question...' and buttons for 'Settings' and a search icon. The footer identifies the tool as 'EviGraph-R · Evidence-grounded research assistant'.

Figure V.10 – Settings panel exposing Top K, score threshold, toggle, and IMRaD section filter.

3.4 Pip Package

To maximise accessibility and encourage open-source adoption, EviGraph-R is distributed as a pip-installable Python package on the Python Package Index (PyPI) under the name `evigraph-r`.

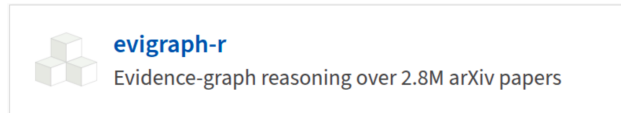


Figure V.11 – EviGraph-R listing on PyPI.

Any researcher or developer can install and query the system with no infrastructure setup:

Listing V.1 – Installing the EviGraph-R Python package

```
pip install evigraph-r
```

Listing V.2 – Querying EviGraph-R through the Python package

```
import asyncio
import evigraph
runner = evigraph.WorkflowRunner()
request = evigraph.QueryRequest(query="What is the effect of BERT pre-
    training on downstream NLP tasks?")
result = asyncio.run(runner.run_query(request))
print(result.answer)
```

The package exposes the full four-agent pipeline as a Python library and provides a command-line interface (`evigraph query / evigraph serve`). By default it connects to the hosted Qdrant vector database on our deployment server, so users can run evidence-graph queries over the 2.28 million arXiv corpus without downloading or indexing any data locally. The source code, issue tracker, and contribution guidelines are publicly available, and the package is released under the MIT licence.

Conclusion

This chapter validated EviGraph-R across retrieval, end-to-end QA, and software integration. The retrieval evaluation showed clear gains over SQuAI. The system evaluation confirmed that each component contributes meaningfully, as shown by the ablation study and baseline comparisons. The resulting system is fully deployable, exposed through a FastAPI backend, a ReactJS interface and a pip package.

Conclusion and Perspectives

In this project, our goal was to address the challenge of reliable, citation-grounded question answering over large-scale scientific literature. We adopted the CRISP-DM methodology, which provided a structured and iterative process, ensuring that each phase of the project was grounded in clear objectives. By following this approach, we successfully designed and deployed EviGraph-R, a multi-agent system capable of answering complex scientific queries with transparent, verifiable evidence. We evaluated the system across three complementary pillars: hybrid retrieval, multi-agent reasoning, and end-to-end answer quality.

Among the architectural choices made, BGE-M3 hybrid retrieval fused via Reciprocal Rank Fusion with cross-encoder reranking emerged as the most effective retrieval strategy. The citation-guided evidence graph and the Judge Agent’s claim verification mechanism further ensured that answers remained faithfully grounded in the retrieved scientific literature. The overall results demonstrated the robustness and reliability of the agentic architecture for real-world scientific question answering.

However, there is ample room for future improvements, which we organize around three strategic directions. From a **data freshness standpoint**, the current index is built on a static snapshot of the unarXive 2024 corpus. A natural and impactful extension would be to establish a continuous ingestion pipeline that periodically fetches newly published articles from arXiv, on a weekly cadence, normalizes, chunks, and embeds them, and incrementally updates the Qdrant vector database. From a **system quality standpoint**, a promising direction is the integration of continuous learning from user feedback. By collecting signals such as query reformulations, citation acceptance or rejection, and explicit relevance judgments through the interface, EviGraph-R could iteratively refine its reranking model, its Judge Agent’s verification thresholds, and its answer generation prompts. From an **integration standpoint**, the Model Context Protocol (MCP) presents a natural and powerful extension point for EviGraph-R. By exposing the system’s core capabilities as MCP-compatible tools, EviGraph-R could be embedded directly into AI-powered environments, coding assistants, and research workflows.

In conclusion, this project demonstrated the effectiveness of a modular multi-agent architecture for reliable scientific question answering at scale. With the proposed extensions: continuous index updates, feedback-driven refinement, and MCP integration, EviGraph-R could evolve into a fully production-grade component of the broader scientific AI infrastructure, where retrieval, reasoning, and verification converge into trustworthy, scholarly-grade systems.

References

- [1] FRAUNHOFER INSTITUTE FOR INDUSTRIAL ENGINEERING IAO. Crisp-dm for surface technology. <https://www.ist.fraunhofer.de/en/expertise/simulation-digital-services/data-acquisition-model-based-process-optimization/crisp-dm-surface-technology.html>. Accessed: 2026-05-12. vi, 7
- [2] ASHISH VASWANI, NOAM SHAZEER, NIKI PARMAR, JAKOB USZKOREIT, LLION JONES, AIDAN N GOMEZ, ŁUKASZ KAISER, AND ILLIA POLOSUKHIN. Attention is all you need. In *Advances in neural information processing systems* (2017). vi, 12
- [3] DR JULIJA. How i built a simple retrieval-augmented generation (rag) pipeline, February (2024). Medium article, accessed 20 May 2026. vi, 13
- [4] RENZO ANGLES. The property graph database model. , **2100** (2018). vi, 15, 16
- [5] ZENTRUM FÜR INFORMATIONSDIENSTE UND HOCHLEISTUNGSRECHNEN (ZIH). Hpc systems. <https://tu-dresden.de/zih/hochleistungsrechnen/hpc>. Accessed: 2025-05-24. vi, 16, 17
- [6] INES BESROUR AND MICHAEL FÄRBER. unarXive 2024: A large-scale scientific corpus for citation-aware retrieval and generation. In *Proceedings of the 2026 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)* (2026). vi, 5, 18, 23
- [7] TUD DRESDEN UNIVERSITY OF TECHNOLOGY. Tud dresden university of technology. https://tu-dresden.de/tu-dresden?set_language=en. Accessed: 2026-05-12. 3
- [8] DRESDEN UNIVERSITY OF TECHNOLOGY. Title of university of excellence. <https://tu-dresden.de/tu-dresden/profil/exzellenz>. Accessed: 2026-05-12. 3
- [9] SCADS.AI DRESDEN/LEIPZIG. About scads.ai dresden/leipzig. <https://scads.ai/about-us/>. Accessed: 2026-05-12. 3
- [10] ARXIV. arxiv. <https://arxiv.org/>. Accessed: 2026-05-12. 4
- [11] PATRICK LEWIS, ETHAN PEREZ, ALEKSANDRA PIKTUS, FABIO PETRONI, VLADIMIR KARPUKHIN, NAMAN GOYAL, HEINRICH KÜTTLER, MIKE LEWIS, WEN-TAU YIH, TIM ROCKTÄSCHEL, SEBASTIAN RIEDEL, AND DOUWE KIELA. Retrieval-augmented generation for knowledge-intensive nlp tasks. , **33**, pages 9459–9474 (2020). 4, 13

-
- [12] INES BESROUR, JINGBO HE, TOBIAS SCHREIEDER, AND MICHAEL FÄRBER. Squai: Scientific question-answering with multi-agent retrieval-augmented generation. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management* (2025). 4, 18, 19, 20, 25, 51, 57
 - [13] PETE CHAPMAN, JULIAN CLINTON, RANDY KERBER, THOMAS KHABAZA, THOMAS REINARTZ, COLIN SHEARER, AND RÜDIGER WIRTH. Crisp-dm 1.0: Step-by-step data mining guide. Technical report The CRISP-DM Consortium (2000). 6, 8
 - [14] USAMA FAYYAD, GREGORY PIATETSKY-SHAPIO, AND PADHRAIC SMYTH. *From data mining to knowledge discovery in databases*. AI Magazine 17(3), 37–54 (1996). 8
 - [15] SAS INSTITUTE. Introduction to semma. <https://documentation.sas.com/doc/en/emref/15.3/n061bzurmej4j3n1jn8bbjjmla2.htm>. Accessed: 2026-05-12. 8
 - [16] ANA ISABEL ROJÃO LOURENÇO AZEVEDO AND MANUEL FILIPE SANTOS. Kdd, semma and crisp-dm: A parallel overview. In *Proceedings of the IADIS European Conference on Data Mining*, pages 182–185 (2008). 8
 - [17] LUCIANA B. SOLLACI AND MAURICIO G. PEREIRA. *The introduction, methods, results, and discussion (imrad) structure: A fifty-year survey*. Journal of the Medical Library Association 92(3), 364–367 (2004). 11
 - [18] ARMAN COHAN, WALEED AMMAR, MADELEINE VAN ZUYLEN, AND FIELD CADY. Structural scaffolds for citation intent classification in scientific publications. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3586–3596. Association for Computational Linguistics (2019). 11, 40
 - [19] TOM B BROWN, BENJAMIN MANN, NICK RYDER, MELANIE SUBBIAH, JARED D KAPLAN, PRAFULLA DHARIWAL, ARVIND NEELAKANTAN, PRANAV SHYAM, GIRISH SASSTRY, AMANDA ASKELL, ET AL. *Language models are few-shot learners*. arXiv preprint arXiv:2005.14165 (2020). 12
 - [20] STEPHEN E. ROBERTSON, STEVE WALKER, SUSAN JONES, MICHELINE HANCOCK-BEAULIEU, AND MIKE GATFORD. Okapi at trec-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, pages 109–126. NIST (1994). 13
 - [21] JIANLV CHEN, SHITAO XIAO, PEITIAN ZHANG, KUN LUO, DEFU LIAN, AND ZHENG LIU. *Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation*. arXiv preprint arXiv:2402.03216 (2024). 13, 14, 26, 37
 - [22] NIKLAS MUENNIGHOFF, NOUAMANE TAZI, LOÏC MAGNE, AND NILS REIMERS. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037. Association for Computational Linguistics (2023). 14

-
- [23] GORDON V. CORMACK, CHARLES L. A. CLARKE, AND STEFAN BUETTCHER. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. Association for Computing Machinery (2009). 14, 38
- [24] NILS REIMERS AND IRYNA GUREVYCH. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, EMNLP '19*, pages 3982–3992. Association for Computational Linguistics (2019). 15, 38
- [25] ADINA WILLIAMS, NIKITA NANGIA, AND SAMUEL R. BOWMAN. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1112–1122. Association for Computational Linguistics (2018). 15
- [26] IBM. What is a multiagent system? <https://www.ibm.com/think/topics/multiagent-system>. Accessed: 2025-05-24. 15
- [27] JUNYOU LI, QIN ZHANG, YANGBIN YU, QIANG FU, AND DEHENG YE. *More agents is all you need*. arXiv preprint arXiv:2402.05120 (2024). 15
- [28] PUPPYGRAPH. Property graph vs RDF: Key differences, working, and use cases, (2025). Accessed May 2026. 15
- [29] TOM SAWYER SOFTWARE. Knowledge graphs, (2023). Accessed May 2026. 16
- [30] GRAPH.BUILD. Introduction to property graphs and LPGs, (2023). Accessed May 2026. 16
- [31] TU DRESDEN. TUD’s capella supercomputer ranks third most powerful supercomputer in Germany. <https://tu-dresden.de/tu-dresden/newsportal/news/tud-hochleistungsrechner-capella>, (2024). Accessed: May 2026. 16
- [32] ZIH, TU DRESDEN. GPU cluster Capella – ZIH HPC compendium. https://compendium.hpc.tu-dresden.de/jobs_and_resources/capella/. Accessed: May 2026. 16, 27
- [33] JOHANNES WELBL, NELSON F. LIU, AND MATT GARDNER. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*, pages 94–106 (2017). 18
- [34] GEORGE TSATSARONIS ET AL. *An overview of the BioASQ large-scale biomedical semantic indexing and question answering competition*. BMC Bioinformatics 16 (2015). 18

-
- [35] KYLE LO, LUCY LU WANG, MARK NEUMANN, RODNEY KINNEY, AND DAN WELD. S2ORC: The semantic scholar open research corpus. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4969–4983 (2020). 18, 23
- [36] JUN ZHAO, CAN ZU, XU HAO, YI LU, WEI HE, YIWEN DING, TAO GUI, QI ZHANG, AND XUANJING HUANG. LONGAGENT: Achieving question answering for 128k-token-long documents through multi-agent collaboration. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 16310–16324, Miami, Florida, USA nov (2024). ACL. 18
- [37] JUNDA ZHU, LINGYONG YAN, HAIBO SHI, DAWEI YIN, AND LEI SHA. ATM: Adversarial tuning multi-agent system makes a robust retrieval-augmented generator. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 10902–10919, Miami, Florida, USA November (2024). ACL. 19
- [38] DIJI YANG, JINMENG RAO, KEZHEN CHEN, XIAOYUAN GUO, YAWEN ZHANG, JIE YANG, AND YI ZHANG. Im-rag: Multi-round retrieval-augmented generation through learning inner monologues. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '24, page 730–740, New York, NY, USA (2024). ACM. 19
- [39] JISOO JANG AND WEN-SYAN LI. Au-rag: Agent-based universal retrieval augmented generation. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, SIGIR-AP 2024, page 2–11, New York, NY, USA (2024). ACM. 19
- [40] YASSER SAEID AND THOMAS KOPINSKI. Agentfusion: A multi-agent approach to accurate text generation. In *2024 International Conference on Electrical and Computer Engineering Researches (ICECER)*, pages 1–8 (2024). 19
- [41] CHIA-YUAN CHANG, ZHIMENG JIANG, VINEETH RAKESH, MENGHAO PAN, CHIN-CHIA MICHAEL YEH, GUANCHU WANG, MINGZHI HU, ZHICHAO XU, YAN ZHENG, MAHASHWETA DAS, AND NA ZOU. Main-rag: Multi-agent filtering retrieval-augmented generation, (2024). 19
- [42] INES BESROUR, JINGBO HE, TOBIAS SCHREIEDER, AND MICHAEL FÄRBER. Ragenta: Multi-agent retrieval-augmented generation for attributed question answering. In *Proceedings of the SIGIR 2025 Workshop on LiveRAG* (2025). Workshop held at the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval. <https://drive.google.com/drive/folders/1ZJk4Z-9JGe72uTWxxPp4IpztTlZy4NFF>. 19
- [43] HANNAH RASHKIN ET AL. 49, pages 131–184 (2023). 19

-
- [44] LUYU GAO ET AL. Enabling large language models to generate text with citations. In *Proceedings of EMNLP*, pages 6465–6488 (2023). [19](#)
- [45] LUYU GAO ET AL. ALCE: An automatic benchmark for large language model generations with citations. In *Proceedings of EMNLP* (2023). [19](#)
- [46] AVIV SLOBODKIN, ERAN HIRSCH, ARIE CATTAN, TAL SCHUSTER, AND IDO DAGAN. Attribute first, then generate: Locally-attributable grounded text generation. In *Proceedings of ACL*, pages 3309–3344 (2024). [19](#)
- [47] PRADEEP DASIGI, KYLE LO, IZ BELTAGY, ARMAN COHAN, NOAH A. SMITH, AND MATT GARDNER. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4599–4610 (2021). [19](#)
- [48] ANIRUDH AJITH, MENGZHOU XIA, ALEXIS CHEVALIER, TANYA GOYAL, DANQI CHEN, AND TIANYU GAO. LitSearch: A retrieval benchmark for scientific literature search. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 15068–15083 (2024). [19](#)
- [49] SÖREN AUER, DANTE A. C. BARONE, CASSIO DANTAS, DENNY VRANDECIC, ET AL. *The SciQA scientific question answering benchmark for scholarly knowledge*. Scientific Reports **13**, 7240 (2023). [19](#)
- [50] PAN LU, SWAROOP MISHRA, TONY XIA, LIANG QIU, KAI-WEI CHANG, SONG-CHUN ZHU, OYVIND TAFJORD, PETER CLARK, AND ASHWIN KALYAN. Learn to explain: Multimodal reasoning via thought chains for science question answering. , **35**, pages 2507–2521 (2022). [19](#), [20](#)
- [51] ANASTASIA KRITHARA, ANASTASIOS NENTIDIS, KONSTANTINOS BOUGIATOTIS, AND GEORGIOS PALIOURAS. *BioASQ-QA: A manually curated corpus for biomedical question answering*. Scientific Data **10**, 170 (2023). [19](#)
- [52] QIAO JIN, BHUWAN DHINGRA, ZHENGPING LIU, WILLIAM W. COHEN, AND XINGHUA LU. PubMedQA: A dataset for biomedical research question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2567–2577 (2019). [19](#)
- [53] JAKUB LÁLA, ODHRAN O'DONOGHUE, ALEKSANDAR SHTEDRITSKI, SAM COX, SAMUEL G. RODRIQUES, AND ANDREW D. WHITE. PaperQA: Retrieval-augmented generative agent for scientific research. In *arXiv preprint arXiv:2312.07559* (2023). [20](#)
- [54] SAMY ATEIA AND UDO KRUSCHWITZ. BioRAGent: A retrieval-augmented generation system for showcasing generative query expansion and domain-specific search for scientific q&a. In *Advances in Information Retrieval: 47th European Conference on Information Retrieval (ECIR)*, pages 1–5. Springer (2025). [20](#)

-
- [55] ELICIT. Elicit: Ai for scientific research. <https://elicit.com/>, (2026). Accessed: 2026-05-20. 20
- [56] CONSENSUS. Consensus: Ai for research. <https://consensus.app/>, (2026). Accessed: 2026-05-20. 20
- [57] SCISPACE. Scispace: Ai for research. <https://scispace.com/>, (2026). Accessed: 2026-05-20. 20
- [58] GREGORY M. KURTZER, VANESSA SOCHAT, AND MICHAEL W. BAUER. *Singularity: Scientific containers for mobility of compute*. PLOS ONE **12**(5) (2017). 27
- [59] VICTOR SANH, LYSANDRE DEBUT, JULIEN CHAUMOND, AND THOMAS WOLF. *Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter*. arXiv preprint arXiv:1910.01108 (2019). 28
- [60] LANGCHAIN. Langgraph documentation. <https://docs.langchain.com/oss/python/langgraph/overview>, (2026). Accessed: 2026-05-29. 36
- [61] PYDANTIC. Pydantic documentation. <https://pydantic.dev/docs/>, (2026). Accessed: 2026-05-29. 36
- [62] STANFORD NLP. Dspy documentation. <https://dsp.ai/>, (2026). Accessed: 2026-05-29. 36
- [63] LANGCHAIN. Langsmith observability documentation. <https://docs.langchain.com/langsmith/observability>, (2026). Accessed: 2026-05-29. 36
- [64] YU. A. MALKOV AND D. A. YASHUNIN. *Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence **42**(4), 824–836 (2020). 37
- [65] IZ BELTAGY, KYLE LO, AND ARMAN COHAN. Scibert: A pretrained language model for scientific text. In *EMNLP*. Association for Computational Linguistics (2019). 40
- [66] MICROSOFT RESEARCH. Claimify: Structured claim decomposition for grounded generation. <https://github.com/microsoft/claimify>, (2024). 41
- [67] ARIC A. HAGBERG, DANIEL A. SCHULT, AND PIETER J. SWART. Exploring network structure, dynamics, and function using NetworkX, (2008). 42
- [68] MAX FRANZ, CHRISTIAN T. LOPES, GERARDO HUCK, YUE DONG, ONUR SUMER, AND GARY D. BADER. *Cytoscape.js: a graph theory library for visualisation and analysis*. Bioinformatics **32**(2), 309–311 (2016). 42
- [69] YUHENG ZHA, YICHI YANG, RUICHEN LI, AND ZHITING HU. AlignScore: Evaluating factual consistency with a unified alignment function. In *Proceedings of ACL 2023* (2023). 43

-
- [70] DASHA METROPOLITANSKY AND JONAH LARSON. *VeriTrail: Closed-domain hallucination detection with traceability*. arXiv preprint arXiv:2505.21786 (2025). [43](#)
- [71] JEFFREY IP AND KRITIN VONGTHONGSRI. DeepEval: The open-source LLM evaluation framework, (2024). [55](#)
- [72] ZHILIN YANG, PENG QI, SAIZHENG ZHANG, YOSHUA BENGIO, WILLIAM W. COHEN, RUSLAN SALAKHUTDINOV, AND CHRISTOPHER D. MANNING. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics (2018). [56](#)
- [73] JOHANNES WELBL, PONTUS STENETORP, AND SEBASTIAN RIEDEL. *Constructing datasets for multi-hop reading comprehension across documents*. Transactions of the Association for Computational Linguistics **6**, 287–302 (2018). [56](#)
- [74] HARSH TRIVEDI, NIRANJAN BALASUBRAMANIAN, TUSHAR KHOT, AND ASHISH SABHARWAL. *MuSiQue: Multihop questions via single-hop question composition*. Transactions of the Association for Computational Linguistics **10**, 539–554 (2022). [56](#)
- [75] VALERIA BOLOTOVA, VLADISLAV BLINOV, FALK SCHOLER, W. BRUCE CROFT, AND MARK SANDERSON. A non-factoid question-answering taxonomy. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1196–1206. ACM (2022). [56](#)